



Sparse Attention Acceleration with Synergistic In-Memory Pruning and On-Chip Recomputation

Amir Yazdanbakhsh* Ashkan Moradifrouzabadi*[†] Zheng Li*[†] Mingu Kang[†]
 Google Research, Brain Team [†]University of California, San Diego (*Equal Contribution)
ayazdan@google.com, ashkan@ucsd.edu, zhengli@ucsd.edu, mingu@ucsd.edu

Abstract—As its core computation, a self-attention mechanism gauges pairwise correlations across the entire input sequence. Despite favorable performance, calculating pairwise correlations is prohibitively costly. While recent work has shown the benefits of runtime pruning of elements with low attention scores, the quadratic complexity of self-attention mechanisms and their on-chip memory capacity demands are overlooked. This work addresses these constraints by architecting an accelerator, called SPRINT¹, which leverages the inherent parallelism of ReRAM crossbar arrays to compute attention scores in an approximate manner. Our design prunes the low attention scores using a lightweight analog thresholding circuitry within ReRAM, enabling SPRINT to fetch only a small subset of relevant data to on-chip memory. To mitigate potential negative repercussions for model accuracy, SPRINT re-computes the attention scores for the few fetched data in digital. The combined in-memory pruning and on-chip recompute of the relevant attention scores enables SPRINT to transform quadratic complexity to a merely linear one. In addition, we identify and leverage a dynamic spatial locality between the adjacent attention operations even after pruning, which eliminates costly yet redundant data fetches. We evaluate our proposed technique on a wide range of state-of-the-art transformer models. On average, SPRINT yields 7.5× speedup and 19.6× energy reduction when total 16KB on-chip memory is used, while virtually on par with iso-accuracy of the baseline models (on average 0.36% degradation).

Keywords—Transformer; Attention Mechanism; Self-Attention; Sparsity; Model Compression; In-Memory Computing; Neural Processing Units; ReRAM; Deep Learning; Hardware-Software Co-Design

I. INTRODUCTION

The sweeping success of self-attention mechanisms shifted the focus of our community from Convolutional Neural Networks [25, 65, 128, 153] to seeking software [70, 71, 75, 76, 138, 163] and hardware approaches [54, 90, 96, 144] to improve efficiency of the attention mechanism. At its crux, it creates and employs three abstractions of its inputs (e.g. words or pixel patches): query, key, and value embeddings. The core operation of self-attention is the computation of pairwise correlations between query and key embeddings, followed by computing a weighted sum of value vectors proportional to measured correlations. Despite its compelling performance, the associated compute and memory footprint cost of self-attention

mechanisms can readily become inordinate², especially as the input sequence length increases (e.g. > 2K), a prevailing trend in recent deep learning models [32, 76, 93, 111, 112, 138].

To address this challenge, a recent line of research [54, 55, 90, 137, 144] intuitively that *each query is germane to only a dynamic subset of the few key embeddings when determining the input context*. This pruning approach appears beneficial, yet does not effectively address the main cost driver of the self-attention mechanism: *data communication overhead*. This is because identifying the relevance of key embeddings per query, especially to preserve model accuracy, still requires fetching all embeddings to on-chip resources and performing costly query–key computations. Commonly, these methods presume sufficiently large on-chip resources to keep all embeddings for a single head on chip. This assumption can readily fail, particularly in models with ever-increasing input sequences and in resource-constrained devices. For example, if we embrace a design with only 20% of requisite on-chip buffers available for embeddings in a head, data communication emerges as the main determinant of efficiency (on average, > 60% of total energy consumption as shown in Figure 1). To address this, we propose in-memory pruning solutions that obviate the need to bring embeddings onto the chip.

An emerging body of work has illustrated significant benefits of ReRAM in-memory computing, due to the inherent efficiency of analog computing and massive parallelism capability [29, 53, 89, 101, 131, 132, 145, 150, 161]. We leverage ReRAM technology to enable in-memory pruning, reducing the pressure on the accelerator to fetch all embeddings onto the chip. While appealing, materializing the possibility of in-memory pruning comes with its own challenges, listed as follows:

- 1) **Circuit inaccuracies:** There are various inaccuracies, such as thermal noise, coupling noise, and process variations associated with ReRAM analog circuitry, which limit the precision of in-memory computing.
- 2) **Data conversion overhead:** Runtime pruning [90], a common approach to preserve model accuracy, requires layer-wise comparisons with a threshold value. The cost of converting the analog results of in-memory computing (multiple bits) to the digital domain for perpetual

¹SPRINT: SParse attention acceleration with appRoximate IN-memory Token pruning

²The cost of pairwise correlations grows in the order of $\mathcal{O}(N^2)$ with respect to input sequence length.

comparisons against threshold values can outweigh the benefits of in-memory computing.

- 3) **Selective read of unpruned embeddings:** Supporting in-memory ReRAM pruning enforces a particular data layout for key embeddings. However, this layout constraints the ability to selectively read the unpruned vectors.

To remedy these considerations, this work makes the following contributions:

- ❶ We introduce a unique perspective on the ReRAM in-memory computing paradigm. We employ approximate in-memory compute and precise on-chip recompute in tandem to mitigate the likely negative repercussions to model accuracy due to inherent circuit inaccuracies.
- ❷ We employ analog comparators to carry out the comparisons with threshold values and instead produce 1-bit data to indicate the pruning status. With this shift in design, we reduce the hardware cost, which is proportional to input bit precision, to merely the cost of a series of 1-bit analog to digital converters (ADCs).
- ❸ We repurpose an existing solution, which enables us to implement data reuse based on our observations. On the hardware side, we rely on recently taped-out transposable ReRAMs [141] that introduce in-situ transposed read access. While initially intended for efficiently accessing neural network weights, our application of this hardware selectively reads unpruned embeddings. For the data reuse, we observe that there is a considerable spatial locality between unpruned key vectors of adjacent queries. We exploit this spatial location to improve data reuse and further reduce the data communication overhead.

We evaluate our approach in several self-attention models with large sequences, including BERT, ALBERT, ViT, GPT-2, and two futuristic designs (e.g. 2K and 4K input sequence length). Under an iso design, our results show that, on average, SPRINT delivers $7.5\times$ speed-up and $19.6\times$ energy reduction compared to a baseline design with 16KB on-chip memory. The benefit increases as on-chip resources become scarcer, representing a design point for resource constrained platforms, e.g. $1.6\times$ more energy reduction with 16KB on-chip memory than the case with 64KB capacity.

II. BACKGROUND AND MOTIVATION

A. Background

Self-attention computations. “Self-attention” computes pairwise correlations across the entire input sequence [140]. Each input element, a word or a pixel patch, is encoded to a vector of size $1 \times e$. We then project these embeddings onto three latent spaces by multiplying each vector into distinct learned weight matrices, W^Q , W^K , and W^V for query, key, and value vectors, respectively. In the next step, we calculate s scores per query vector q_i (i -th row out of s rows in Q), each score representing the relevance of a query to all the key vectors, including itself. The resulting scores are then normalized by employing a row-wise “Softmax” operation,

producing an attention probability matrix. A higher probability value indicates greater relevance of the corresponding element with respect to others in the input sequence.

Finally, to obtain the attention values ($A_{s \times d}$), the probability matrix is multiplied by the value matrix followed by a sum reduction. Intuitively, the objective of this step is to scratch out the value of the low probability elements, while intensifying the others. To further improve the performance of the self-attention mechanism, multiple paths with dedicated query, key, and value weight matrices are introduced. This self-attention is generally known as “multi-headed”. Under this paradigm, final values are generated via a concatenated form of attention vectors from each head, which is projected onto a matrix of size $s \times d_w$ using one or more feed-forward layers.

Learned runtime pruning. The self-attention mechanism is prohibitively costly in terms of computation and memory, in the order of $\mathcal{O}(s^2)$. Recently proposed methods [54, 90, 144] prune attention values with low scores by banking on inherently large redundancy in input sequences. While [54, 144] trades model accuracy for higher performance, LEOPARD [90] proposes a learned runtime pruning method trailed by an early compute termination mechanism to ensure on par model accuracy with baseline. Once complete, it incorporates the learned threshold values during inference to prune inconsequential scores, after performing the entire computation of $Q \times K^T$, cutting down most of the computations after Softmax. A common theme among existing methods is the assumption of sufficiently large on-chip buffers to store the entire key and value matrices. However, this assumption fails at longer input sequences [32, 76, 93, 112, 138] (e.g. $> 2K$) as well as for resource-constrained accelerators. This work builds upon the learned runtime pruning method introduced in [75, 90] and specifically tackles the pressures on on-chip capacity and data communication overhead. Our objective is to eliminate unnecessary data communications and on-chip computations of $Q \times K^T$ by approximating the thresholding mechanism inside memory.

B. Motivation

As the input sequence length is poised to increase dramatically for future transformer models (e.g. $s = 1024$ and 2048 in GPT [111]), motivated by the resulting improvements in model performance [32, 76, 93], the assumption of sufficient on-chip resources is no longer valid. Additionally, as transformer models pave their way into resource-constrained devices [155], the increased demand for on-chip memory capacity and higher compute efficiency form a challenging design target, even for transformer models with modest sequence length (e.g. $s = [128, 256, 384]$ for BERT [71, 82]).

While recent literature [54, 90, 144] favors pruning, these approaches nevertheless are plagued by the considerable overhead of data communication even with adequately sized on-chip buffers. This cost is exacerbated when on-chip resources are limited, because of frequent instances of data communica-

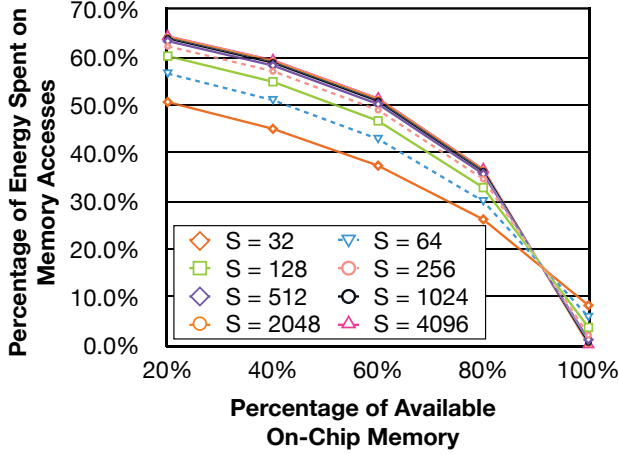


Figure 1: Percentage of energy spent on memory accesses to process one attention head with respect to various percentages of available on-chip memory. The results are shown across various sequence (S) length.

tion. Figure 1 measures the contribution of off-chip memory read and write accesses to the overall energy consumption to process a single-head self-attention layer³. The x-axis encompasses various fractions of on-chip memory capacity with respect to different input sequence lengths. As on-chip resources become scarce (20% of requisite on-chip buffers available to store the entire key and value matrices), on average, the energy contribution of on-chip memory increases to $> 60\%$, turning into the dominant energy contributor. In this tightly-budgeted scenario, approaches that unlock the opportunity to fetch only a subset of relevant data become attractive.

One such compelling solution is applying the run-time pruning such as [54, 90, 96, 144]. However, even these techniques require to bring in the entire key and value matrices to exercise thresholding. This research tackles above challenge by approximating $\mathcal{Q} \times \mathcal{K}^T$, followed by a comparison with threshold values. Despite the approximation, our results ensure that this in-memory thresholding mechanism can consistently identify the entire subset of relevant vectors. To guarantee accuracy on par with baseline, we recompute the score values in a precise manner after selective data fetching.

C. Data Communication Optimization

Processing self-attention scores with limited on-chip memory capacity requires frequent data movement between adjacent query vectors. This section points out several opportunities to cut down the cost of such movements.

1) In-memory Thresholding

Under scarce on-chip resources, a logical optimization step can leverage in-memory computing to eliminate inconsequential data communications for pruned key and value vectors. For example, in Figure 2, the core simply stipulates $\mathcal{K}_{2,4,5,6,11,13}$ for $q_1 \times \mathcal{K}^T$ computations ($q_1 \rightarrow \text{"The"}$). This

³Section VII outlines the experimental setup details.

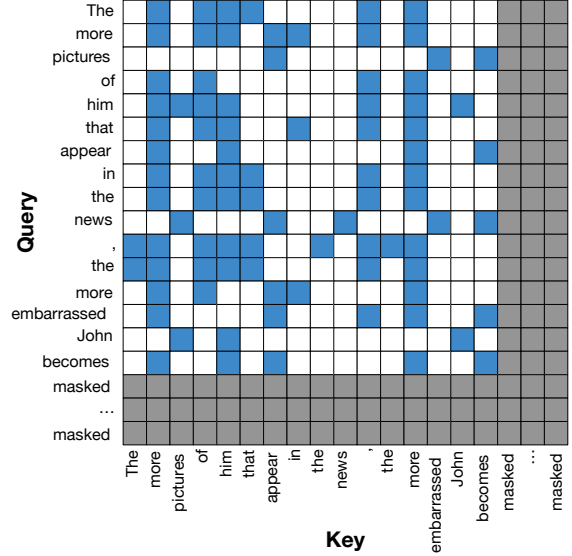


Figure 2: Query-Key relation for the first attention layer of CoLA task from GLUE dataset [143]. White squares represent pruned entries. The gray stripes are masked regions.

observation provides the opportunity to significantly cut costs by informing the accelerator to only fetch the requisite data.

2) Spatial Locality in Adjacent Queries

While in-memory thresholding trims down the amount of data per query that are brought into on-chip buffers, it increases the frequency of data fetches. This is because a new set of key and value vectors should be fetched to proceed computing for subsequent queries once the computations for $q_i \times \mathcal{K}^T$ completes. This increase in the frequency of data fetches may well neutralize the potential benefits of reducing the amount of transferred data.

To explore future potential reductions in the amount of transferred data and compensate for the likely overhead of frequent data transfers, we study the similarities between unpruned keys across input queries. Figure 2 illustrates a real example of CoLA task from GLUE dataset [143] (eighth head in the first attention layer). Each row indicates a query and its corresponding unpruned key locations, filled in blue. The grey shading on the last few rows and columns specifies the input mask, commonly used in transformer models when the sequence length in the input dataset is less than the one in the model. It is visually evident that a significant number of keys are inconsequential per query, and that there is a high spatial locality between adjacent rows. For example, compared to query “The”, the additional required keys for the adjacent query “more” are only “appear” and “in”. The remaining unpruned key elements, such as “more”, “of”, and “him”, are identical between these queries, obviating additional data transfers.

Theoretical expectation of spatial locality. Equation 1 calculates the probability of $\mathcal{L}$, defined as the number of

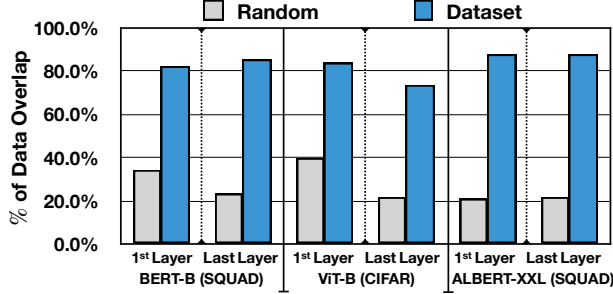


Figure 3: Number of common indices between neighboring tokens (Q_i vs. Q_{i+1}) with the practical dataset vs. randomly selected pruned tokens with the pruning rate from [90].

overlapping elements between adjacent queries of size S . In this equation, M represents the number of the unpruned elements in each query. The probability of $\mathcal{L}$ is calculated by first multiplying the numbers of possible combinations of $\mathcal{L}$ elements out of M and the remaining $M - \mathcal{L}$ elements out of $S - M$. This product is subsequently divided by the number of possible combinations of M elements out of S . The resulting probability of each $\mathcal{L}$ is then multiplied by the value of $\mathcal{L}$ and summed across M to calculate the theoretical expected overlap between adjacent queries, as demonstrated in Equation 1.

$$P(\mathcal{L}) = \frac{M C_{\mathcal{L}} \times S - M C_{M-\mathcal{L}}}{S C_M}, \quad E(\mathcal{L}) = \sum_{\mathcal{L}=1}^M \mathcal{L} \cdot P(\mathcal{L}) \quad (1)$$

In Figure 3, we compare the percentages of overlaps, averaged across multiple inputs and observed in various extant datasets [39, 71, 111], with the theoretical expectation formula, as presented in Equation 1. The results reveals a striking 2 - 3 $\times$ increase in the observed overlap percentage in the real world scenarios. This increase highlights a notable data reuse opportunity because most of the requisite elements already reside in on-chip buffers. Therefore, exploiting this data reuse opportunity limits the number of data fetches only to the unpruned elements that differ between adjacent queries, leading to a dramatic cost reduction. One could leverage spatial locality across larger windows (>2) at the cost of hardware complexity. However, on average, the resulting overlap is below 5%, which does not justify the requisite overhead.

3) Futile Computations in Padded Regions

It is a common practice [148] in transformer models to pad input sequences that are shorter than the maximum supported length. The padded inputs do not meaningfully contribute to the self-attention computations, and hence are irrelevant for the final model accuracy. These padded regions are highlighted as gray squares in Figure 2, where only 16 queries out of 128 are computationally relevant. This leaves $(128-16) \times (128-16)$ score computations inconsequential. The padded regions are commonly nullified by placing a sufficiently large negative value [148]. Passing these negative values through Softmax prompts their probability to approach

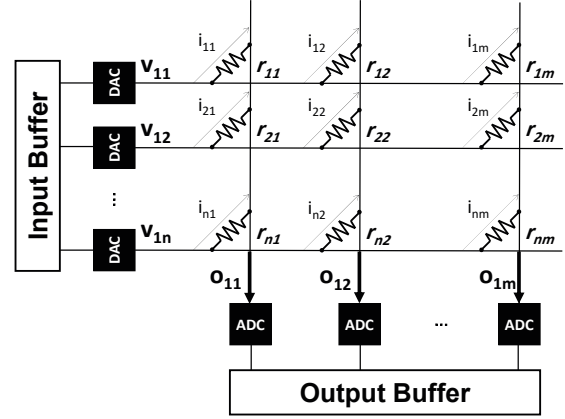


Figure 4: In-memory computing with ReRAM cross-bar array.

zero, excluding them from subsequent computations. To further eliminate unnecessary data communications in these padded regions, we can proactively identify them as early as possible in memory.

III. IN-MEMORY THRESHOLDING

Overview of ReRAM. Resistive Random Access Memory (ReRAM) is a non-volatile memory that stores data using its adjustable resistance. Figure 4 demonstrates a ReRAM 2D crossbar array [102]. To further improve the density and energy efficiency of ReRAM, recent methods [95, 149, 160, 165] use Multi-Level Cells (MLC) to store multiple bits of information inside each cell. In contrast to Single-Level Cells (SLC), the MLC ReRAM permits a range of resistance values inside each cell. Although storing more bits per cell appeals by increasing ReRAM memory density, it can easily become a limiting factor. As the number of bits/cell increases, each cell renders itself more amenable to circuit noises and limits the accuracy of computations. Recent studies [15, 60] deem a four bits/cell MLC ReRAM design the optimal balance between robustness and complexity of current sensing detection circuitry.

Vector-Matrix multiplication with ReRAM in-memory computing. ReRAM can perform efficient and highly parallel analog vector-matrix multiplications, as demonstrated by prior work [29, 53, 123, 131] on DNN acceleration. To perform such multiplications, the matrix elements are mapped onto memristor conductance and the input vector is fed into ReRAM's wordlines (Figure 4, horizontal lines), one element per row, as biased voltages generated by a digital-to-analog converter (DAC). Additionally, a sum reduction can be executed on the resulting multiplications across the crossbar columns as serial currents [89, 151]. Once complete, the weighted-sum vector forms an analog current at the boundary of the ReRAM crossbar, one element per column. The

following equation formally presents a multiplication between vector $v_{1 \times n}$ and matrix $M_{n \times m}$ on a ReRAM crossbar array:

$$m_{ij} = \frac{1}{r_{ij}} \quad o_{1j} = \sum_{i=1}^n v_{1i} \cdot m_{ij} \quad (2)$$

where m_{ij} and r_{ij} represent each element of matrix M and its corresponding resistance value in ReRAM cells.

Application in run-time pruning. The in-memory principle introduced above can be seamlessly applied for accelerating the attention mechanism. This can be achieved by storing each k_i vector in a column of the crossbar array, and applying the input voltage level, which corresponds to the element of query vector q_i , to each wordline as described in Figure 6(a). Ideally, we require s columns to store entire sequence length while d rows are needed to accommodate the entire embedding size. If the array size does not match with problem size, multiple banks of array can be employed in a tiled manner. All of k_i vectors stored in multiple columns are processed for parallel dot-product operations in one shot. Once it completes, the next query vector q_{i+1} is processed in the subsequent cycle.

Analog↔Digital challenges. It is shown [29, 89, 123] that digital↔analog conversion drains a significant portion of total ReRAM power consumption, especially as the number of conversion bits increases. For example, the power and area of a 5-bit ADC are $>20\times$ [139] and $>30\times$ [136] higher than a 1-bit ADC, respectively. Therefore, it is crucial to take the power overhead of these converters into account, especially for designs with greater than one-bit precision requirements. In the following, we discuss the main challenges to in-memory thresholding.

A. In-Memory Thresholding Challenges

① Analog computing inaccuracies. Analog computing in ReRAM is commonly known to be susceptible to inherent circuit noises and inaccuracies, such as thermal noise, temperature fluctuations, process variations, and coupling noise between adjacent cells [23, 59, 64]. These inaccuracies limit the feasible precision of computations in ReRAM crossbar arrays. To evaluate the impact of limited compute precision for in-memory thresholding on the final model accuracy, we use the following approach⁴:

$$\text{Prune} = \text{Argwhere}(\text{Score}_R^b < \mathcal{T}h); \text{Score}[\text{Prune}] = -c \quad (3)$$

where Score_R^b denotes the in-memory score values (e.g. results of $q_i \times \mathcal{K}^T$) when the output has limited accuracy with a b -bit precision. “Argwhere” finds the indices of score elements that are lower than the target threshold. Note that the threshold values ($\mathcal{T}h$) are learned during the full-precision finetuning process such as LEOPARD [75, 90]. The scores of the identified pruning indices are then forcefully set to a large negative value ($-c$) to remove irrelevant elements.

⁴As we explain in Section VII, we do not perform additional fine-tuning to quantize key values to lower bit-precision.

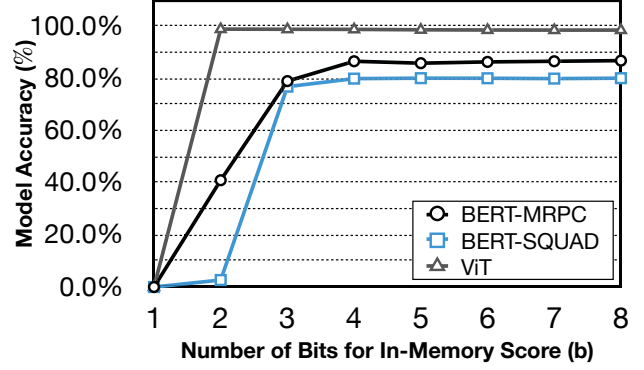


Figure 5: Sensitivity of model accuracy to the number of bits (b) used for in-memory thresholding (comparison of in-memory scores with $\mathcal{T}h$, Equation 3).

Also, recall that we perform low-precision in-memory computing for the sole purpose of identifying the irrelevant key vectors. With on-chip accelerators, the score computation for unpruned vectors is still performed in full-precision.

Figure 5 compares the final model accuracy after quantizing the Score with different bit-precision (b) across three different models: BERT-Base [71] with GLUE [143], BERT-Base with SQUAD [113], and ViT [39] with [78] dataset. The results show that the quantization error with 4-bit precision virtually has no impact on the final model accuracy⁵. Thus, the runtime pruning mechanism is robust against approximation, even when the computation has a certain level of errors. This is intuitive because the incorrectly pruned vectors already exhibit a small score value, likely in the vicinity of $\mathcal{T}h$. Hence, the impact on model accuracy is negligible. Finally, even more sensitive workload to the noise can be in theory compensated by adding a modest negative margin on top of $\mathcal{T}h$ in Equation 3 at the cost of reducing the pruning ratio (directly proportional to hardware performance).

② ADC converter overhead. The overhead of ADC converters increases proportionately to the precision of conversion. Two design choices can support comparisons between vector-matrix multiplication outputs and the threshold values. The first option uses a 5-bit ADC to convert the outputs and employs digital comparators for thresholding. The other option utilizes analog comparators for thresholding prior to ADC. The output of each analog comparison represents a binary value, which indicates whether to prune the corresponding key vector. Since the resulting pruning vector only requires one bit per key, we can use a low-overhead 1-bit ADC (implemented as a comparator). The low overhead of 1-bit ADC ($>20\times$ [139] lower area and $>30\times$ [136] lower power consumption compared to a 5-bit ADC) favors the second option for in-memory thresholding.

③ Reading unpruned vectors overhead. Finally, performing

⁵A recent study from HP Lab [60] has shown that ReRAM in-memory computing for 64-tap dot-product delivers 5-bit equivalent output accuracy after including all the error sources.

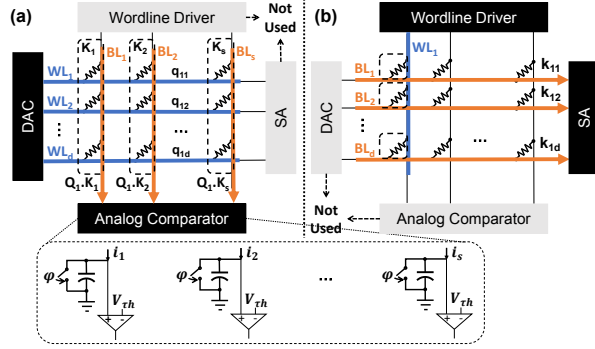


Figure 6: Transposable ReRAM crossbar array. (a) ReRAM crossbar during in-memory pruning, (b) Transposed ReRAM crossbar during normal read.

in-memory thresholding followed by fetching each unpruned $\mathcal{K}$ vector from ReRAM arrays (for digital re-compute) is arduous and can impose significant read latency. This occurs because we store each vector of $\mathcal{K}$ vertically at each ReRAM column (Figure 4), and k_i is mapped to the i^{th} ReRAM column. On the other hand, accessing from ReRAM through a standard read operations fetches the data stored horizontally in a row. Therefore, fetching from ReRAM requires sequentially asserting *all* the (horizontal) wordlines, bringing in each row of the $\mathcal{K}$ matrix (even the ones associated with *pruned* k vectors), and selectively fetching the unpruned vectors to on-chip buffers. We address this challenge by a recent taped-out transposable ReRAM proposal [141], which we expound below.

B. Transposable ReRAM for Thresholding

Overview. A transposable ReRAM [141] supports (1) in-situ access to the array to perform vector-matrix computations (in-situ computation), as well as (2) reading *their transposed* values (transposed read). Figure 6 shows the overall design of a transposable ReRAM in these two modes. In the “in-situ computation” mode, the ReRAM array performs vector-matrix multiplications, similarly to conventional (non-transposable) ReRAM crossbar shown in Figure 4. In this case, we assign the value of each element in input vector q_i to wordlines (horizontal) and assert all the bitlines (vertical) to enable parallel multiplications. On the other hand, in the new “transposed read” mode, the horizontal lines become bitlines and vertical line becomes wordline. In this mode, *only* one wordline gets asserted. Once the bitline current from all the columns are fully developed, the sense amplifier reads all the values stored on the ReRAM conductance of the asserted wordline (in the column).

In-memory thresholding dataflow. As discussed in the previous section, one of the challenges for performing in-memory thresholding is reading unpruned vectors after score calculation (3). The “transposed read” mode presents a viable solution to this challenge. Next, we present a dataflow to identify unpruned key vectors leveraging

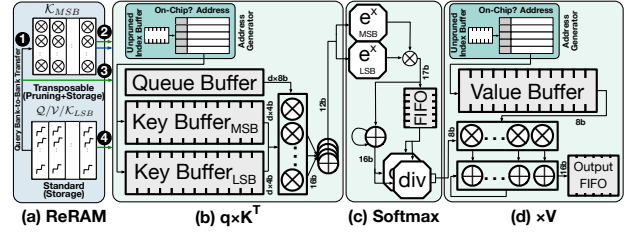


Figure 7: The overview of SPRINT system. Data transfers within ReRAMs and between ReRAMs and accelerator consist of: (1) bank-to-bank between standard and transposable ReRAMs, (2) unpruned $\mathcal{K}_{MSB}$ vectors and their corresponding indices, (3) Ω vector, and (4) unpruned $\mathcal{K}_{LSB}$ and $\mathcal{V}$ vectors.

transposable ReRAMs. In this dataflow, we store each key vector vertically in the ReRAM crossbar array (the first key vector is mapped onto the first ReRAM column, and so on). Because analog circuit noises limit the supported bit-precision on each memory cell, we *only* store a predefined subset of MSB-side bits within each cell. Our experiment showed that a 4-bit precision is sufficient for in-memory thresholding, yielding on par model accuracy. As such, we only store four MSBs per key vector element in transposable ReRAM arrays. The rest of LSBs can be stored on conventional ReRAM modules. Similarly, the elements of query and value vectors are stored on conventional ReRAM modules. Note that these modules do not need any support for in-memory computations and are solely used for storage⁶.

To process the query vector $q_{1 \times S}$, the on-chip accelerator first transmits a subset of query vector MSBs to the transposable ReRAMs⁷ that store the key matrix $\mathcal{K}_{d \times S}^T$. A low-precision DAC converts the digital values of the query vector to analog and feeds them into the ReRAM via wordlines. The transposable ReRAM array performs a low-precision vector-matrix multiplication in analog to calculate the Scores, which are produced after vertically applying an analog reduction sum per key vector. The next step performs in-memory thresholding using analog comparators. Note that the threshold values can either be set at the start of the computations or sent along with each q vector. Finally, after performing the analog comparisons, a voltage value (corresponding to a 1-bit digital value) flows through a series of 1-bit ADCs. The ADC outputs indicate the pruning state of their corresponding key vectors, where “1” means pruned.

The generated binary pruning vector is sent back to the on-chip accelerator, which subsequently gets translated into multiple memory requests to selectively fetch unpruned key and value vectors from their corresponding modules. Note that the pruning vectors for both key and value are completely identical. Upon receiving the first unpruned key vector, the

⁶We homogeneously use ReRAM for storage of queries and values and in-memory thresholding for simplicity. Another possibility can exploit a heterogeneous design, in which DRAM memories are used for query/value matrices and small ReRAM crossbar arrays for in-memory thresholding.

⁷The number of MSBs in query and key are identically set to 4-bits.

accelerator can start recomputing Scores in full-precision. The same process repeats for the rest of the query vectors.

IV. OVERVIEW OF SPRINT SYSTEM

As shown in Figure 7, the overall SPRINT architecture includes two main components: 1) ReRAM memory, and 2) on-chip accelerator, as described below.

ReRAM memory. ReRAM memory banks are split into two categories, standard and transposable. Standard ReRAM is solely used for storage ($\mathcal{Q}$, $\mathcal{V}$, and $\mathcal{K}_{LSB}$), while transposable ReRAM is for both storage ($\mathcal{K}_{MSB}$) and performing dot-product and in-memory thresholding, informing the on-chip accelerator which embedding vector to fetch. The in-memory thresholding mechanism, explained in Section III, exclusively uses MSBs to determine pruning criteria.

On-chip accelerator. The SPRINT on-chip accelerator performs three main operations, $q \times \mathcal{K}^T$, Softmax, and $\times \mathcal{V}$ in a pipelined manner. Figure 7 depicts the major microarchitectural units with their associated bit precision and data flow. The arithmetic is further described in Section VI. The accelerator fetches the unpruned k / v vectors from ReRAM along with a binary vector, indicating which k / v indices are unpruned, to store in unpruned index buffers. Based on these indices, the address generator block produces addresses to access the unpruned vector from $\mathcal{K}/\mathcal{V}$ buffers. The accelerator first performs $q_{1 \times d} \times \{\mathcal{K}_{MSB}, \mathcal{K}_{LSB}\}$ followed by an adder tree to precisely calculate the score values. Note that, $\mathcal{Q}$ buffer only stores the streamed-in $q_{1 \times d}$ temporarily for the window of score computation. Then, the Softmax block normalizes the Score values into a probability distribution proportional to the exponentials of the input Scores. Finally, the last block multiplies each v vector by their corresponding Score probability, followed by a reduction sum across the weighted v vectors to generate the final attention values.

V. SPRINT MEMORY CONTROLLER

Background. A memory controller receives a stream of memory access requests from the on-chip accelerator, generates their corresponding memory command stream. The memory controller consequently arbitrates the memory commands and schedules them to off-chip memory according to a scheduling policy. The technology of a memory (e.g. DRAM or ReRAM) dictates a set of timing constraints that must be satisfied by the memory controller between each issued memory command. To communicate data between the on-chip accelerator and off-chip memory, a sequence of memory commands generated by the memory controller are required. These commands collectively retrieve data from rows across multiple chips into their corresponding row buffers and select a column from the currently fetched retrieved data. A subsequent column access to the same row enjoys the row-buffer locality, hence, lowest access latency. However, the consecutive accesses between different

rows are generally suffer from substantially higher access latency. The memory controller aims to schedule the memory commands in order to maximize the row-buffer locality.

A. Data Layout Organization

We presume a similar organization as conventional memory subsystems for SPRINT. In general, optimizing the data layout organization for deep learning applications is straightforward because of their predictable memory access pattern. We observe the same pattern for SPRINT data layout organization. As explained, to support in-memory thresholding, we presume a non-interleaving data organization for $\mathcal{K}$ s (similar to prior work [41, 74, 84]). That is, we store each vector of k (a column in $\mathcal{K}_{d \times s}^T$) in one column of memory mat. Based on our observation (spatial locality between unpruned key indices, Section II-B), we distribute the neighboring k vectors across different banks/channels. Our empirical results show that this distribution of k vectors provides a better utilization of memory bandwidth and reduces structural conflicts. Same data layout organization works for v vectors. The $\mathcal{Q}$ matrix, on the other hand, does not need to follow this particular data layout organization. That is because each q vector is processed sequentially and after every $q \cdot \mathcal{K}$ vector-matrix multiplication which provides sufficient time for the memory subsystem to handle the upcoming query read requests.

The final data layout organization requirement is for the MSB and LSB parts of k vectors. As described in Section III-B, MSB and LSB parts of key vectors must be distributed across different type of ReRAM crossbar arrays, transposable and conventional respectively. This separation of MSB and LSB bits can be established statically before the computation starts. To effectively enable this special data layout organization, we can provide device-side allocation APIs so the user can specify different requirements for $\mathcal{Q} / \mathcal{K} / \mathcal{V}$ matrices without exposing physical underlying structure of memory subsystem. Similar software support has been proposed in prior work [31, 74].

Scaling for embedding size. One potential challenge to the proposed data layout organization and in-memory thresholding mechanism is posed by scalability. Specifically, as the embedding size of key vectors increases, applying the reduction sum across each column of ReRAM arrays may seem infeasible. This limitation can be readily addressed by splitting the key vector into multiple adjacent ReRAM columns, similarly to [67]. With this circuit modification, the resulting analog current from the adjacent key vector splits can be subsequently merged and compared with the threshold value.

B. Memory Controller Microarchitecture

The on-chip memory controller designed for SPRINT is separated into a frontend and a backend engine. The frontend engine communicates with multiple on-chip accelerators, accepting memory requests, whereas the backend engine generates and issues commands to off-chip memory modules with respect to their timing constraints.

C. Memory Controller Execution Flow

Overview. The memory controller in SPRINT governs the tasks of in-memory thresholding and fetching the corresponding unpruned $d \times 1$ vectors of $\mathcal{K}_{d \times s}^T$ matrix. To complete these operations, the memory controller first sends a low-precision variant of q_i vector of size $1 \times d$ to $\mathcal{K}_{MSB}$ ReRAM banks. Each $\mathcal{K}_{MSB}$ ReRAM bank executes low-precision in-memory thresholding and generates a binary pruning vector of size s . The j^{th} element of the generated binary vector indicates whether to prune the j^{th} column of $\mathcal{K}_{d \times s}^T$ matrix (i.e., '1' $\rightarrow$ pruned and '0' $\rightarrow$ unpruned). Upon receiving the binary pruning vector, the memory controller processes this vector and consequently issues a stream of read requests to fetch the unpruned vectors of $\mathcal{K}_{d \times s}^T$ matrix.

Spatial locality detection engine. To further reduce the data movement between off-chip memory and on-chip buffers, we design and integrate a spatial locality detection (SLD) engine in the front-end of the memory controller. The primary task of the engine is to detect and exploit spatial locality between the last and current binary pruning vectors associated with the attention score computations for adjacent query vectors (i.e., $q_{1 \times d}^i$ and $q_{1 \times d}^{i+1}$). The advantages are two folds: (1) "only" generating memory requests for k vectors that do not exist in on-chip $\mathcal{K}$ buffer, hence reducing data transfer and memory contention, and (2) bootstrapping the attention score ($Q \times K^T$) computations for the k vectors that already reside in on-chip $\mathcal{K}$ buffer, hence minimizing the data transfer latency. The following equations describe the logic behind these two tasks given the last and current binary pruning vector:

$$\text{Task 1} \rightarrow \text{Memory Requests Vector} = \mathcal{P}_{1 \times s}^{t-1} \wedge \overline{\mathcal{P}_{1 \times s}^t} \quad (4)$$

$$\text{Task 2} \rightarrow \text{Spatial Locality Vector} = \overline{\mathcal{P}_{1 \times s}^{t-1}} \wedge \mathcal{P}_{1 \times s}^t \quad (5)$$

where $\mathcal{P}_{1 \times s}^{t-1}$ and $\mathcal{P}_{1 \times s}^t$ represent binary pruning vectors associated with the last and current attention score computations at a given time point t , respectively.

Memory request generator engine. The main objective for the memory request generator (MRG) engine is to produce a potentially limited number of memory requests to fetch key vectors that do not currently reside in on-chip key memory. Each memory controller retains one MRG engine to produce the corresponding key vector addresses residing in that particular bank. At each cycle, a binary value is read from the memory request vector. If zero, it means that the corresponding key vector is not required for the current attention score computation; hence, bypassing memory request generation step. On the other hand, a one-value indicates that a key vector must be fetched from off-chip memory. Hence, a memory request with an address corresponding to the location of the desired key vector is generated.

To satisfy the key vector organization requirement (Section IV), we decided to statically place the adjacent key vectors into memory modules attached to different channels. As such, to properly generate the key vector addresses,

we equip each MRG with a base register and a shared up counter block. The base register indicates the starting key vector index located on a particular memory channel. The up counter starts from zero upon receiving a binary pruning vector and increases by the number of memory channels. We also equip each memory controller with a key index generator (KIG) engine, which has the exact same microarchitecture. However, in lieu of memory request vector, KIG engine operates on spatial locality vectors to generate the key vector addresses for SPRINT on-chip engines in order to bootstrap the attention score computations.

Memory commands and timing considerations. Supporting SPRINT style in-memory thresholding into memory requires introducing additional memory commands and memory timing constraints. To enable in-memory thresholding in SPRINT, we introduce two additional memory commands, CopyQ and ReadP. CopyQ copies elements of query vector to in-memory query buffer, whereas ReadP reads elements of resulting binary pruning vector from in-memory pruning vector buffer. Depending on the bit-width of query and pruning vectors, the memory controller may issue one or more consecutive CopyQ and ReadP commands. Note that to initiate in-memory thresholding computations, we add one-bit in CopyQ command in which a one-value indicating the start of computations. Issuing other memory commands will be prohibited amid in-memory thresholding computations.

As you may observe, there is some similarities between CopyQ and ReadP commands and normal memory read and write, respectively, projecting a similar timing constraints as read/write commands. However, since CopyQ works with an isolated buffer from memory arrays, it neither requires tRP for row pre-charging, nor tRCD to activate a memory row. On the other hand, since consecutive CopyQ commands still occupy data buses, we adhere to the tCL timing constraint. The scenario for ReadP is quite different as it communicates with the bank row buffers to read the resulting binary pruning vectors into on-chip buffers for further processing. Therefore, we conservatively follow the exact same timing constraints as memory read command for ReadP. For both introduced commands, burst CopyQ and ReadP follow the same timing constraints as normal burst memory read and write.

While the described scenarios for CopyQ and ReadP covers most of the required timing constraints, it still leaves one crucial timing constraints between adjacent CopyQ and ReadP commands. This timing, dubbed tAxTh, represents the number of cycles that each ReRAM crossbar requires to perform in-memory thresholding and producing the resulting pruning vector. Our circuit simulations show that this timing is < 8 cycles [21].

Power implications of in-memory thresholding. In addition to timing constraints, memory systems are also under power budget limitations. tFAW and tRRD represent the memory timing constraints linked to power budget.

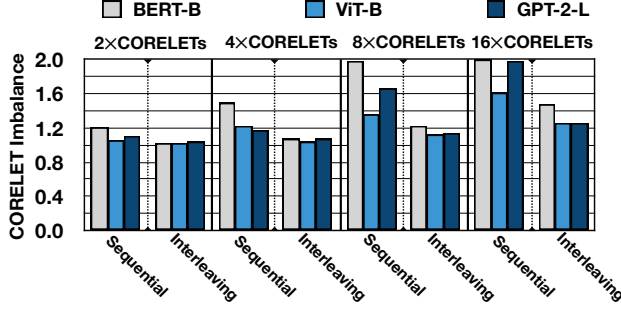


Figure 8: CORELET utilization imbalance with and without token interleaving across CORELETs.

To account for this power budget limitation, we model the analog in-memory thresholding circuit and estimate the power of analog comparators. Our simulation shows that the overhead of additional analog circuitry for analog comparisons merely increases the total power budget by $< 0.07\%$ of total in-memory computation [89]. This power overhead has negligible implications on these timing constraints, hence we posit the nominal values for tFAW and tRRD in our simulations (similar to work [72, 74]).

VI. SPRINT ON-CHIP ACCELERATOR

The SPRINT processor includes N CORELETs to enable a higher parallelism degree. A CORELET is an independent processing block that computes the entire self-attention mechanism pipeline, including $Q \times K^T$, Softmax, and $\times V$. Each CORELET consists of a QK-processing unit (QK-PU) and a V-processing unit (V-PU). QK-PU performs the $1 \times d$ dot product between q and k , whereas V-PU processes the $1 \times d$ dot product between the Softmax output and v in the digital domain. In addition, each CORELET has a small number of buffers to store unpruned key and value vectors. Note that the query vectors are processed in a stream manner, and thus do not need multi-entry buffers (Q-buf). Finally, each CORELET has its own look-up-tables to record which key and value vectors are currently present on chip.

Workload balancing across CORELETs. SPRINT accelerator can simultaneously process multiple key vector sub-elements in each CORELET while the same query vector is distributed among all the CORELETs. As soon as the computations of one query and all of its associated keys complete, the computations of the next query can begin. In this design, the adjacent key vectors are assigned to different CORELETs, called *token-interleaving*. For example, given total four available CORELETs, SPRINT process $\mathcal{K}_{4n+i}$ in the i -th CORELET if the token is unpruned. This balances the workload across CORELETs while considering the spatial locality, by which the unpruned indices tend to appear in adjacent locations.

Figure 8 shows the workload imbalance ratio with $2\times$, $4\times$, $8\times$, and $16\times$ CORELETs. We calculate the imbalance ratio by dividing the maximum by the minimum numbers of assigned

unpruned tokens per CORELET and averaging the numbers for all the queries (i.e. the value of one implies ideal workload balance across CORELETs). The proposed workload distribution scheme considerably improves the utilization balance compared to the sequential token mapping, e.g. neighboring tokens belonging to the same CORELETs. We observe that for large models such as GPT-2-L, SPRINT can readily leverage higher parallelism from more CORELETs. However, SPRINT may underutilize the CORELETs for smaller models (e.g., BERT-B) because of their inferior parallelism opportunity.

Handling data misses. To minimize the number of stalls due to data misses, the unpruned key vectors are proactively prefetched by the memory controller (as explained in Section V-B). We also configure the main memory bandwidth (Table I) to provide a new pair of k and v in burst mode to further reduce such stalls. Note that by leveraging the spatial locality between unpruned key vectors, on average, only 2.1% of the sequence length is required to be fetched between adjacent queries. This high data reuse drastically reduces the likelihood of data misses. When a rare data miss occurs, the computations for the next available key vector can proceed until the data miss is handled by the memory controller. We implement this bypassing of unavailable key vectors by adding a rotating pointer to key/value index buffers.

SPRINT accelerator arithmetic operations. Once at least one key vector resides in K-buf, the computation can start. At each cycle, SPRINT performs a dot-product between each subset of elements from key and query vectors. If all the key elements can not be processed during one compute iteration, SPRINT stores the partial sums in a register until the results are ready to be processed by a Softmax module. Similar to prior work [54, 90], we use a two look-up-tables method for exponent calculation. Afterwards, SPRINT stores the streaming outputs in FIFOs for accumulation. Once complete, each score is normalized to produce the corresponding probabilities. To balance the throughput between different stages of the pipeline, we employ two divider units. Finally, the computed score probabilities are used in V-PU to calculate the weighted sum of v vectors. Note that the unpruned indices for key vectors can be used for the pruning of value vectors as well.

Two-dimensional sequence reduction. As introduced in Section II-C3, a large portion (e.g. 46% for the SQUAD dataset) of the total sequence length is futile due to zero-padding. Figure 2 illustrates the zero-padded (gray) area, which reduces the required output computation in both vertical and horizontal dimensions. Horizontally, the computation is reduced to k vectors per q , whereas vertically, it is reduced to q vectors. We implement this mechanism by enabling the memory controller to filter out the read requests for these masked regions.

SPRINT accelerator design choice. The SPRINT accelerator does not employ a double-buffering scheme for on-chip memory in order to avoid the doubled cost of memory capacity. When the new data arrives from main memory,

those are stored in a temporary small buffer. Meanwhile, a stall request is issued to initiate the write process into K-buf and V-buf. Note that, due to spatial locality across unpruned k elements for adjacent q vectors, the number of newly fetched k / v is infrequent. Similar to prior work [54, 90, 144], SPRINT performs all the computations in 8-bit precision, except Softmax with 12-bit inputs. For final attention score, we employ 16-bit precision.

VII. METHODOLOGY AND EVALUATION

Benchmarks. We use the following models to evaluate the efficacy of SPRINT: BERT-Base (BERT-B) [71], BERT-Large (BERT-L) [71], ALBERT-X-Large (ALBERT-XL) [82], ALBERT-XX-Large (ALBERT-XXL) [82], ViT-Base (ViT-B) [24], and GPT-2-Large (GPT-2-L) [111]. We employ the Stanford Question Answering Dataset (SQUAD) [113] to test BERT-B, BERT-L, ALBERT-XL, ALBERT-XXL, the WikiText-2 [99] to test GPT-2-L, and CIFAR10 [78] dataset to test the ViT-B. We use the default sequence length (s) of 197, 384, and 1024 for the CIFAR10, SQUAD and WikiText-2 datasets, respectively. All models use an embedding size of $d=64$. On top of the above datasets, we create two additional synthetic models Synth1 and Synth2 with 2K and 4K sequences. These additional models estimate the projected benefit of SPRINT architecture for longer input sequences.

Model fine-tuning for target benchmarks. For the baseline, we use pre-trained models from HuggingFace [148] and fine-tune them on each task with the reported hyperparameters [39, 71, 82]. We only alter batch size due to our limited GPU memory. Nonetheless, even with reduced batch size, the final accuracy after fine-tuning does not change discernibly. Following the described methodology [90], we implement differentiable soft thresholding into the studied transformer models. We identify the optimal pruning threshold per attention layer as part of the task-specific fine-tuning process. We use identical hyperparameters for training, except for the learning rate and the number of epochs. The search space for the model learning rate is $\{2e^{-3}, 2e^{-4}, 2e^{-5}\}$, whereas the search space of $\{2e^{-5}, 2e^{-6}\}$ is used for the learned threshold. The number of epochs varies from one to three depending on the target tasks. We conduct our experiments with PyTorch v1.10 [106] on an Nvidia RTX 3090 GPU, except for GPT-2-L, for which we use an Nvidia A100 GPU.

The resulting pruning rates for BERT-B, BERT-L, ALBERT-XL, ALBERT-XXL, ViT-B, and GPT-2-L are 74.6%, 75.5%, 65.1%, 73.1%, 64.4%, and 73.9%, respectively. For Synth1 and Synth2, we set a pruning rate of 75% and a padding ratio of 50%. The estimated main memory access when switching to new query vector for Synth1 and Synth2 are obtained by scaling up the numbers from BERT-B based on the sequence length difference.

SPRINT hardware simulations. Table I lists the design parameters of SPRINT for three studied configurations: (1)

Table I: Hardware configurations of SPRINT.

Modules	Configurations for S-SPRINT / M-SPRINT / L-SPRINT
ReRAM BW	16×64-bit channels @ 1 GHz per CORELET
ReRAM Array	256 × 128 standard bitcell, 64 × 128 transposable array with 4-b MLC
On-Chip Cache	16/32/64KB in total of K/V buffers (= 8/16/32 banks), 128-b port per bank
QK-PU / V-PU	1/2/4 EA of 1-D 64 (=D) way 8×8-b MAC array
Softmax	1/2/4 EA of 12-b input, 8-b output, 2EA of 64B LUTs, 2EA of dividers
Query Buffer	64B / 128B / 256B
Index Buffer	0.5KB / 1KB / 2KB

Table II: Energy consumption of major microarchitectural units of SPRINT.

Microarchitecture Units	Energy
QK-PU/V-PU Dot-Product	192.56 (pJ); 8 bits, 64-tap
Key/Value Buffer	256 (pJ); 4 banks with 128-bit access per bank
Softmax	89.8 (pJ); 2 LUT accesses + multiply + division
Analog Comparator	5.34 (pJ); 128 Columns
In-Memory Computation	833.6 (pJ); 64 Rows×128 Columns
ReRAM Access	Write: 12492.8 (pJ), Read: 1587.2 (pJ); 512 bits

S-SPRINT: a CORELET with 16KB, (2) M-SPRINT: two CORELETs with 32KB, and (3) L-SPRINT: four CORELETs with 64KB total on-chip buffer capacity. We use Cadence Genus 19.1 [19] for the logic synthesis and Cadence Innovus 19.1 [20] for the placement/routing (PnR) of digital blocks with a 65 nm TSMC general-purpose standard cell library. We generate the digital blocks to meet the target frequency of 1 GHz from the post-layout simulations. For SRAM on-chip memories, we use ARM Memory Compiler with High density 65 nm single-port SRAM (version r0p0) [13] to measure its energy consumption. ReRAM crossbar in-memory operation consumes 0.10 pJ/MAC in 65 nm including the digital-to-analog conversion (DAC) [21]. The standard ReRAM read/write operations consume 3.1 pJ/bit and 24.4 pJ/bit, respectively [51]⁸. Each analog comparator consumes 41 fJ [89]. A recent study of ReRAM in-memory computing [60] has shown that 64-tap in-memory dot-product delivers 5-bit equivalent output accuracy. To emulate the limited accuracy of the in-memory thresholding, we use an identical error specification in Section III-A with $b=5$.

SPRINT performance simulator. We collect the numbers of in-memory dot-product operations and analog comparisons, read accesses to ReRAM, on-chip $q \times \mathcal{K}^T$ for unpruned elements, accesses to LUTs, and division operations for Softmax. In addition, we compile the numbers of additions and multiplications to calculate the weighted sum of v vectors. For the numbers of read accesses to ReRAM, the simulator properly accounts for the spatial locality between adjacent queries and limited on-chip memory capacity. Finally, because the majority of these statistics are input-dependent, we average these numbers across the entire input dataset for each model. For energy consumption, we multiply the average number of operations across the self-attention layers by their corresponding energy consumption from post-layout

⁸Compared to NVSim [38] with a similar configuration, we use a more conservative model with 1.6× and 7.2× higher read delay and energy, respectively, to accommodate for additional ReRAM overheads.

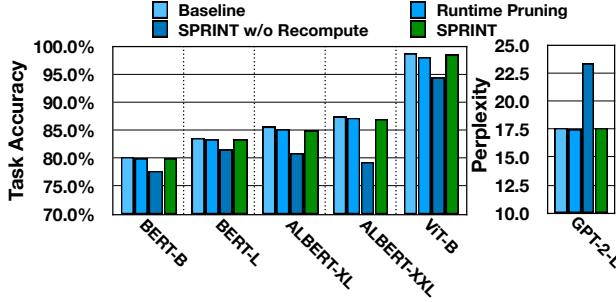


Figure 9: Comparison of task accuracy between baseline and different SPRINT configurations. Second and third bars show task accuracy with runtime pruning and SPRINT w/o on-chip recompute. Fourth bar depicts SPRINT accuracy after including on-chip recompute. GPT-2-L accuracy is measured as a perplexity metric (lower is better).

simulation along with the reported access energy for ReRAM, as listed in Table II. For latency and throughput estimation, we calculate the delay of self-attention layers while taking into account the in-memory thresholding compute delay along with the communication delay between ReRAM arrays and CORELETs. We also implement token interleaving (See Section VI), which distributes tokens across the entire input dataset to CORELETs. Such interleaving better leverages the spatial localities between k vectors and minimizes the imbalance factor. We report the delay of each self-attention layer as the worst-case delay across the N CORELETs.

Baseline architecture. We employ the same configuration of S-SPRINT, M-SPRINT, and L-SPRINT, but without the in-memory pruning, proposed memory controller, and two-dimensional computing reduction for the padded sequences. We compare SPRINT and the baseline at iso-setups including the same frequency, the number of processing elements, on-chip memory capacity, and bit widths for all the input and output of digital logic blocks.

Comparison to prior systems. A^3 [54], SpAtten [144], and LeOPard [90] also support the run-timing pruning to minimize the required computation. A^3 is a prior state of the art on using approximation to accelerate attention mechanism. A^3 thresholds after processing a limited number of k vectors from the sorted queue in a magnitude order to minimize the run-timing pruning overhead. Nonetheless, A^3 does not consider the data movement cost from the main memory assuming enough on-chip memory capacity. LeOPard performs the gradient-based training to co-optimize the model accuracy and pruning rate by tuning the pruning threshold automatically during the training instead of empirical methods. Again, LeOPard does not consider the cost from main memory access. SpAtten proposed a cascaded pruning to exclude the redundant heads and tokens from all the subsequent layers once those are pruned in the previous layer. SpAtten reduces the DRAM access cost for GPT-2, but not for other models assuming enough on-chip capacity.

A. Accuracy and Performance

Impacts on model accuracy from in-memory pruning.

Figure 9 depicts the model accuracy of the various models under four different scenarios: (1) baseline (software-only) [148], (2) with runtime pruning, (3) SPRINT without on-chip recompute, and (4) SPRINT that includes both in-memory thresholding and on-chip recompute. On average, the absolute accuracy difference (excluding GPT-2-L) for runtime pruning (second bar) and SPRINT (fourth bar) is 0.22% with maximum degradation of 0.24% in ALBERT-XL. Compared to runtime pruning, SPRINT improves ViT-B accuracy by 0.5%. Figure 9 also ablates the impact of on-chip recompute (third bar) on accuracy. On average, the accuracy degradation of SPRINT without on-chip recompute is $\approx 4\%$ (5.71 perplexity gap in GPT-2-L). On average, compared to the baseline models (first bar), accuracy degradation of SPRINT is merely 0.36% and separately, the perplexity of GPT-2-L increases by 0.10. These results underscore the importance of on-chip recompute in preserving the baseline model accuracy.

Main memory data movement analysis. Figure 10 shows the reduction in the total amount of data movement from the main memory to the processor (compared to S-Baseline) during processing a single self-attention head. We illustrate the data movement reduction in two configurations: (1) “Mask Only” $\mapsto$ sequence reduction for the padded area and (2) “SPRINT” $\mapsto$ run-timing pruning on top of the sequence reduction of the padded area. We normalize the results to S-Baseline, in which neither of these optimizations are employed. The data reduction is higher with L-SPRINT due to the large on-chip buffer whereas S-SPRINT requires more data movement. Across the 48 studied configurations, our proposed system yields, on average, 94.9%, 98.5%, and 98.9% in S-SPRINT, M-SPRINT and L-SPRINT, respectively. The benefit varies across workloads due to their different pruning rate and the portion of padded area. For instance, BERT-B has higher data movement reduction due to its 46% padded area and 74.6% pruning rate compared to ViT with 64.3% pruning rate and no padded area. The mask only scenario has the modest data movement reduction of 65.2%, 84.5%, and 92.2% in S-SPRINT, M-SPRINT, and L-SPRINT, respectively. The only exception is observed in ViT-B due to the lack of zero padding. Specifically, ViT-B is an exception because M-SPRINT has already sufficient memory capacity to store entire (197) sequence length. The gap among S-, M-, L-SPRINT configurations is the narrowest in Synth1 and Synth2, where the input sequence length is significantly larger. Thus, even L-SPRINT model can accommodate only highly limited fraction of the entire sequence length, e.g. 12.5% in Synth2. For the same reason, the data movement reduction is less significant in Synth models compared to others as those cannot contain enough number of correlated tokens in their scarce on-chip memory.

Performance and energy comparison. Figure 11 compares the SPRINT speedup over the baseline design across all the

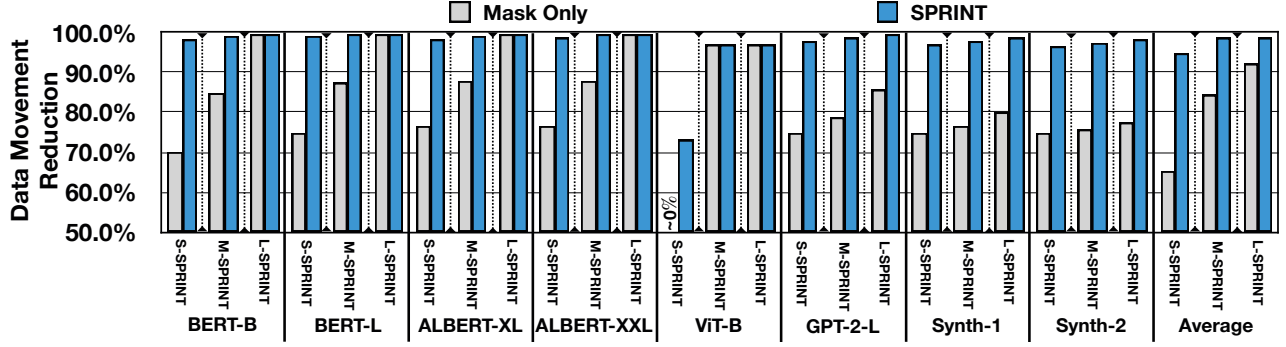


Figure 10: Total data movement reduction from main memory normalized to that of S-Baseline configuration in two scenarios: (1) “mask only” $\rightarrow$ with sequence reduction for the padded area and (2) “SPRINT” $\rightarrow$ with run-time pruning on top of the sequence reduction.

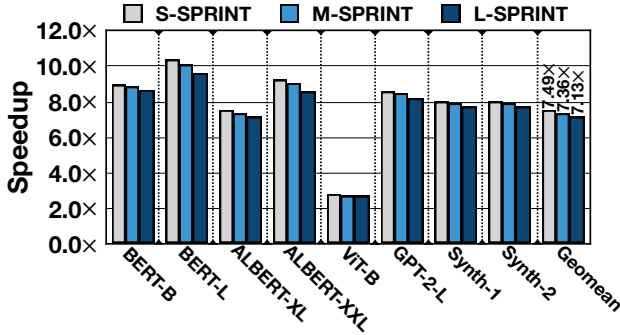


Figure 11: Speedup comparison to a baseline design for self-attention layers.

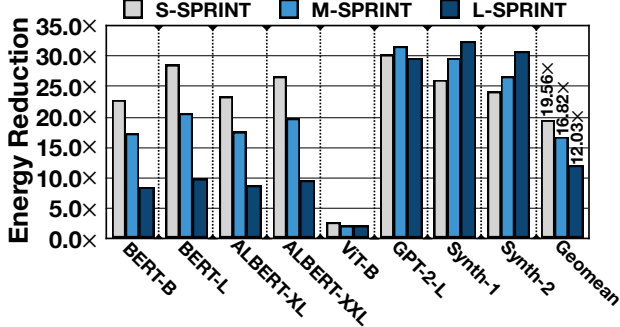


Figure 12: Total energy reduction compared to a baseline design for self-attention layers.

24 studied task. On average, S-, M-, and L-SPRINT achieve 7.5 $\times$, 7.4 $\times$, and 7.1 $\times$ speedup, respectively. These speedups are attributed to skipping the majority of computing cycles due to the in-memory run-time pruning. From the ablation study, the limited speedup of 1.8 $\times$, 1.7 $\times$, and 1.7 $\times$ is achieved on average from the run-time pruning without the in-memory computing support. This is because all the $\mathcal{Q} \times \mathcal{K}^T$ must initially be computed in the on-chip accelerator so the $\times \mathcal{V}$ processing can be pruned. The speedup benefit diminishes in L-SPRINT slightly (<5% compared to S-SPRINT) because the workload utilization is

not appropriately balanced across CORELETs (See Figure 8) even after k vector distribution. BERT-L enjoys the maximum benefits with 9.6 - 10.4 $\times$ speedup while ViT-B has minimum improvement of 2.7 - 2.8 $\times$. This is because of the different pruning rates and portion of padded area in those models.

Figure 12 shows the energy reductions achieved by SPRINT, including on-chip accelerator and ReRAM-based main memory, compared to the Baseline for the three configurations. We observe an energy reductions of 19.6 $\times$ for S-SPRINT, 16.8 $\times$ for M-SPRINT, and 12.0 $\times$ for L-SPRINT. The S-SPRINT achieves the largest energy reduction because the proportion of main memory access out of the total energy is significantly higher than the other configurations. We attribute this higher main memory accesses to the highly constrained memory capacity and frequent memory accesses. This leads to more improvement by the proposed technique which reduces the data movement effectively. On the other hand, Synth1 and Synth2 show the exception in this trend because even L-SPRINT can contain only very few fraction of the entire sequence, e.g. 12.5% in Synth2 vs. 100% in BERT-B. In such a regime, where the memory capacity is significantly limited, the larger memory provides more room to fetch the correlated data together increasing the chance of data re-use. Therefore, L-SPRINT achieves more energy benefit compared to S- and M-SPRINT in Synth models. The energy benefit is greater in Synth1 and Synth2 models than the other cases as those require more frequent data access due to their large sequence length so that the benefit by SPRINT is magnified. In contrast, ViT-B shows the minimum benefit due to its small sequence length, and thus infrequent data access.

Energy consumption breakdown. Figure 13a details the energy breakdown of M-SPRINT with pruning-only (second bar) and with pruning+in-ReRAM thresholding (third bar). The energy breakdown includes: (1) ReRAM read/write, (2) in-ReRAM pruning, (3) on-chip $\mathcal{K}/\mathcal{V}$ buffers read/write, and (4) computations in QK-PU, Softmax, and V-PU. On average, in baseline (first bar) 47.8% of the energy consumption comes from ReRAM read operations, except ViT-B, whose input sequence is 2 $\times$ - 5 $\times$ shorter compared to other models.

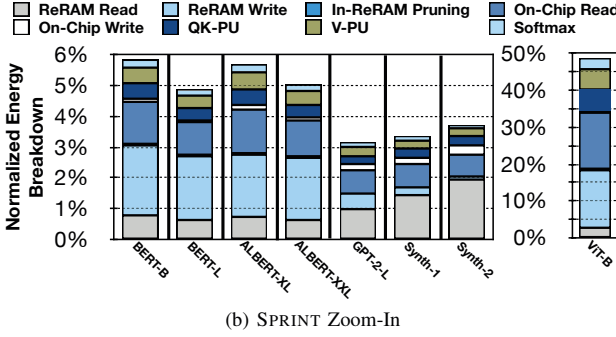
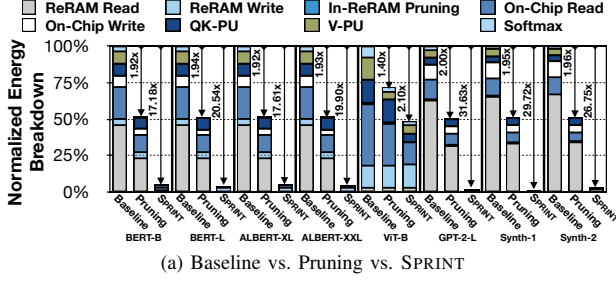


Figure 13: M-SPRINT’s energy breakdown normalized to baseline for self-attention layers. (a) The first bar shows the energy breakdown for baseline (no pruning), whereas the second and the third bars present the energy breakdown for pruning-only and SPRINT (in-ReRAM pruning), respectively. (b) Zoomed-in view of normalized energy breakdown for SPRINT.

For the pruning-only scenario, SPRINT still needs to fetch the entire q and k vectors from ReRAM, even though some are inconsequential, and perform the requisite $Q \times K^T$ computations followed by on-chip comparison with threshold values for the run-time pruning. After that, Softmax and $\times V$ are processed for the unpruned tokens. Therefore, most benefits in the pruning-only case originate from reducing the number of main memory and on-chip memory reads for V and operations in Softmax and dot-product in V-PU. Across the self-attention models, we observe around $1.9\times$ – $2.0\times$ energy savings, except ViT with only $1.4\times$. We attribute the lower energy savings in ViT-B to its lower pruning rate (64%), fewer spatial localities (on average $2.6\times$ less compared to other models), and lack of masking (See the gray stripes in Figure 2).

With pruning and in-ReRAM threshold (third bar), SPRINT significantly reduces the number of ReRAM reads as well as on-chip computations ($Q \times K^T$, Softmax, and $\times V$), mainly by the virtue of in-ReRAM pruning. In this configuration (zoomed-in view in Figure 13b), SPRINT only fetches the elements of K and V matrices (unpruned ones) that certainly contribute to the computation of final attention values. On average, SPRINT reduces the overall energy consumption of self-attention layers by $16.9\times$. Compared to other models, ViT-B confers significantly lower energy savings, merely $2.10\times$, for the same reasons as the pruning-only scenario. While SPRINT architecture slightly reduces the number of ReRAM writes by obviating this need for zero-padded areas,

Table III: SPRINT performance comparison with prior work. SPRINT and LeOPard use the 65 nm technology node, whereas A³ and SpAtten use the 40 nm version.

Metric (unit)	A ³	SpAtten	LeOPard	M-SPRINT
Sequence Length	50 - 384	384 - 1024	50 - 1024	128 - 4096
Process (nm)	40	40	65	65
Area (mm ²)	2.1	1.6	3.5	1.9
Key Buffer (KB)	20	24	48	16
Value Buffer (KB)	20	24	64	16
GOPs / s	518.0	360.0	574.1	1816.2
GOPs / J	4709.1	382.0	519.3	902.7
GOPs / s / mm ²	249.0	238.0	165.5	973.5
GOPs / s / J / mm ²	2263.6	252.5	119.7	469.7
Mem. Cost Included	✗	✓	✗	✓

ReRAM writes still have the highest contribution to the overall energy consumption. The overhead of in-memory pruning including peripheral circuitry is negligible (only 4%) due to its highly parallel and low-voltage analog operations. These results substantiate that the in-ReRAM pruning benefits outweigh its marginal overhead.

Data movement cost analysis. The data movement described in Figure 7 is categorized (1) bank-to-bank transfer of Q (1 in Figure 7), and the transfer from the ReRAM main memory to the on-chip accelerator for (2) Q (3), and (3) unpruned K (2, 4) and V (4). We include the contributions of these data movements in the analysis of Figure 12 and Figure 13. The energy overhead of (1) is negligible ($<0.04\%$ of the entire energy based on the post-layout simulations with an activity factor of 0.5) as it is the intra-chip data transfer. However, on average, (2) takes $<3.7\%$ whereas (3) consumes 31.2% , 20.7% , and 15.5% of energy for S-/M-/L-SPRINT, respectively.

Comparison with A³, SpAtten, and LeOPard. Table III lists the details of prior works and M-SPRINT architecture in terms of throughput (GOPs / s), energy efficiency (GOPs / J), and area efficiency (GOPs / s / mm²). For fair comparison, we also included the area from the in-memory thresholding [141], which takes only 3% out of total M-SPRINT area. Due to the absence of reported results in A³, we calculated above results in Table III given the frequency and power numbers obtained from [54]. The prior arts considered the scenario of enough on-chip memory with minimal consideration of the dram access cost. On the other hand, SPRINT includes all the costs from the frequent main memory access assuming the limited on-chip memory scenario by considering $> 4\times$ longer sequences (up to 4096) than prior arts.

Compared to prior work, M-SPRINT yields the best GOPs / s and GOPs / s / mm² even including the main memory access cost due to its in-memory pruning. Compared to A³, M-SPRINT achieves $3.5\times$ and $3.9\times$ improvements in GOPs / s and GOPs / s / mm² respectively. However, it achieves $5.2\times$ lower GOPs / J. This is due to two reasons: (1) the DRAM access read and write costs are not considered in the results of A³ and (2) the lower process technology (40

nm) in A^3 . Taking into account the difference in the process technology node (65 nm vs. 45 nm), GOPs / J of SPRINT increase to 3873.5 with Dennard scaling [37] ($1.2\times$ lower than A^3). Moreover, M-SPRINT achieves $3.2\times$ higher GOPs / s and $5.9\times$ higher GOPs / s / mm^2 than LeOPard. Although the DRAM access costs are not incorporated in LeOPard, M-SPRINT still delivers $1.7\times$ higher GOPs / J. Finally, M-SPRINT achieves $5.0\times$, $2.4\times$, and $4.1\times$ enhancements in GOPs / s, GOPs / J, and GOPs / s / mm^2 , respectively, as compared to SpAtten. The benefits that are gained from GOPs / s and GOPs / s / mm^2 are based on the early stage in-memory pruning by leveraging the spatial locality.

Table III also compares GOPs / s / J / mm^2 between M-SPRINT and prior work [54, 90, 144]. M-SPRINT yields $1.9\times$ and $3.9\times$ higher GOPs / s / J / mm^2 compared to SpAtten and LeOPard, respectively. However, M-SPRINT delivers $4.8\times$ lower GOPs / s / J / mm^2 compared to A^3 , mainly due to considering DRAM access read and write costs and lower process technology node. With Dennard scaling [37], GOPs / s / J / mm^2 of SPRINT increase to 8648.5 ($3.8\times$ better than A^3).

End-to-End comparison including fully-connected networks (FFNs). Although SPRINT focuses on accelerating self-attention layers, the proposed accelerator can be repurposed to perform the FFN by exploiting QK/V-PU as two 8-bit input 64-tap dot-product engines. The $\mathcal{K}/\mathcal{V}$ buffers store 16KB weights of the FFN to provide 128 8-b weights per cycle by reusing the weights over many inputs. The M-SPRINT achieves speed and energy benefits for the end-to-end execution even in such small benchmarks (BERT-B: $2.2\times$ / $1.8\times$, BERT-L: $2.4\times$ / $2.0\times$ for energy saving / speedup) by avoiding futile computations for the padded region (see Section II-C3), effectively reducing the iterations in FFN computations. ViT-B achieves only marginal benefit ($1.1\times$ / $1.0\times$) due to the lack of padded area. M-SPRINT achieves greater benefit for larger benchmarks, e.g. $7.7\times$ / $4.7\times$ for Synth2.

SPRINT on-chip accelerator and ReRAM in-memory Area. Figure 14 shows the S-SPRINT layout in a 65 nm process which occupies $1.18 \times 0.8 \text{ mm}^2$ including 16KB on-chip SRAM. The layout estimation of ReRAM in-memory [141], including 64×128 transposable array and other peripheral circuitry, is also shown in Figure 14. Due to the inherent high-density of ReRAM, the area overhead takes only around 6% in S-SPRINT.

VIII. RELATED WORK

Contrary to the broad spectrum of in-memory computing [14, 69, 74, 77, 97, 103, 107], SPRINT principally positions itself as a joint in-memory analog pruning and on-chip digital recomputation system for attention-based models. This synergistic method yields substantial gains and curtails the costly on-chip memory requirement. These gains are maintained while preserving the baseline model

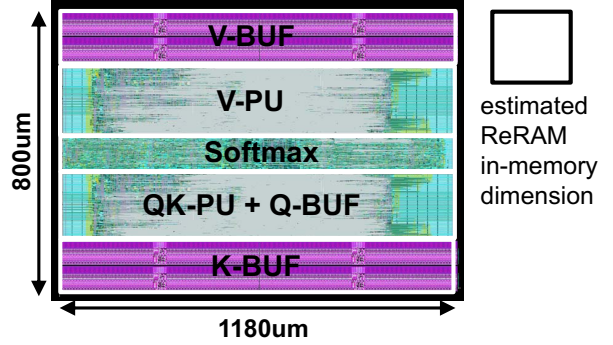


Figure 14: S-SPRINT on-chip accelerator layout with estimated ReRAM in-memory area overhead [141].

accuracy. We review the relevant literature here.

In-memory computing and specialized memory controller design. We can broadly categorize in-memory computing into (1) 3D stacking [3, 4, 92, 108, 164, 166, 169], (2) exploiting the inherent massive parallelism inside memory [10, 14, 44, 74, 84, 84, 98, 120–122, 156], and (3) emerging memory technologies and DRAM modifications [1, 2, 2, 11, 12, 17, 29, 36, 43, 52, 53, 62, 63, 68, 81, 87, 88, 123, 129, 134, 141, 142, 159]. Similar to this prior work, SPRINT also exploits the internal structure of memory to enable a form of in-memory computation. However, our work distinguishes itself by seamlessly blending lightweight in-memory analog approximate computing and on-chip precise recompute. The first phase informs the on-chip accelerator to only fetch a few relevant key vectors from memory, reducing the hefty cost of data communication, while the second phase ensures model accuracy on par with baseline models. SPRINT also intersects with [5, 6, 18, 22, 47, 57, 58, 86, 114] as it similarly equips the memory controller with custom hardware blocks and specialized memory commands to unlock the full potential of in-memory thresholding.

Machine learning acceleration. There is an abundance of prior work on accelerators for machine learning [8, 9, 26–29, 35, 40, 42, 45, 46, 48–50, 54–56, 61, 65, 66, 73, 79, 80, 85, 90, 94, 96, 104, 105, 109, 115, 116, 118, 119, 123–128, 131, 133, 135, 137, 144, 152–155, 162, 167]. SPRINT explores a different design point by seamlessly combining in-memory thresholding and on-chip recomputation to reduce the costly data communication overhead. In addition, there is a line of work on software-only techniques that statically induce sparsity in self-attention [16, 30, 76, 100, 110, 117, 146, 147, 157, 158]. On the other hand, recent work [33, 34, 168] unlocks dynamic sparsity in self-attention models, yet provoking the entire computation of $Q \times K^T$. Finally, a class of hardware-software methods targets early compute termination [7, 83, 91, 130]. While this work is not closely related, our contribution of disseminating the computations between in-memory and on-chip can be employed to perform

in-memory identification of early compute opportunities.

IX. CONCLUSION

Self-attention mechanisms have become integral to transformer models in multiple applications, ranging from natural language processing to computer vision. Despite their benefits, attention mechanisms require extravagant compute and storage space resources, quadratically proportional to input sequence length. Recent work has presented the benefits of runtime pruning in self-attention mechanisms, albeit overlooked quadratic complexity and on-chip memory capacity requirements. SPRINT harnesses the inherent parallelism of ReRAM crossbar arrays to compute the attention scores in a low-precision format. The resulting attention scores cross a lightweight analog thresholding circuitry, which dynamically prunes the inconsequential scores. Hence, SPRINT fetches only a small subset of relevant data to on-chip memory. To mitigate the negative repercussion of approximate ReRAM computations on model accuracy, SPRINT recomputes the sparse attention scores for the few fetched data in digital. Furthermore, the paper identified and exploited a dynamic spatial locality between the adjacent attention operations even after runtime pruning. This spatial locality further reduces the redundant data fetches and scales down the on-chip memory demand. The combined in-memory pruning and on-chip recomputation of the relevant attention scores reduce the quadratic complexity of self-attention mechanism into a merely linear one. The proposed shift in compute and space complexity yields significant performance gains as well as enables the acceleration of futuristic models with significantly larger input sequence length.

ACKNOWLEDGEMENTS

We would like to extend our gratitude towards Hadi Esmailzadeh, Soroush Ghodrati, Stella Aslibekyan, Suvinay Subramanian, James Laudon, and extended Google Research, Brain Team for their invaluable feedback and comments.

REFERENCES

- [1] S. Aga, S. Jeloka, A. Subramanian, S. Narayanasamy, D. Blaauw, and R. Das, "Compute Caches," in *HPCA*, 2017.
- [2] A. Agrawal, A. Jaiswal, D. Roy, B. Han, G. Srinivasan, A. Ankit, and K. Roy, "Xcel-RAM: Accelerating Binary Neural Networks in High-Throughput SRAM Compute Arrays," *IEEE Transactions on Circuits and Systems I*, 2019.
- [3] J. Ahn, S. Hong, S. Yoo, O. Mutlu, and K. Choi, "A Scalable Processing-in-Memory Accelerator for Parallel Graph Processing," in *ISCA*, 2015.
- [4] J. Ahn, S. Yoo, O. Mutlu, and K. Choi, "PIM-enabled Instructions: A Low-Overhead, Locality-Aware Processing-in-Memory Architecture," in *ISCA*, 2015.
- [5] B. Akesson, "An Introduction to SDRAM and Memory Controllers," <https://www.es.ele.tue.nl/~premadona/files/akesson01.pdf>, 2018, accessed: 2021-04-18.
- [6] B. Akesson, K. Goossens, and M. Ringhofer, "Predator: A Predictable SDRAM Memory Controller," in *CODES+ISSS*, 2007.
- [7] V. Aklaghi, A. Yazdanbakhsh, K. Samadi, H. Esmailzadeh, and R. K. Gupta, "SnaPEA: Predictive Early Activation for Reducing Computation in Deep Convolutional Neural Networks," in *ISCA*, 2018.
- [8] J. Albericio, A. Delmás, P. Judd, S. Sharify, G. O'Leary, R. Genov, and A. Moshovos, "Bit-Pragmatic Deep Neural Network Computing," in *MICRO*, 2017.
- [9] J. Albericio, P. Judd, T. Hetherington, T. Aamodt, N. E. Jerger, and A. Moshovos, "Cnvlutin: Ineffectual-Neuron-Free Deep Neural Network Computing," in *ISCA*, 2016.
- [10] S. Angizi and D. Fan, "ReDRAM: A Reconfigurable Processing-In-DRAM Platform for Accelerating Bulk Bit-Wise Operations," in *ICCAD*, 2019.
- [11] S. Angizi, Z. He, and D. Fan, "DIMA: A Depthwise CNN In-Memory Accelerator," in *ICCAD*, 2018.
- [12] S. Angizi, Z. He, and D. Fan, "ParaPIM: A Parallel Processing-in-memory Accelerator for Binary-Weight Deep Neural Networks," in *ASPAC*, 2019.
- [13] ARM, "Artisan Memory Compilers," <https://developer.arm.com/ip-products/physical-ip/embedded-memory>, 2021, accessed: 2021-11-08.
- [14] B. Asgari, R. Hadidi, J. Cao, D. E. Shim, S.-K. Lim, and H. Kim, "Fafnir: Accelerating Sparse Gathering by Using Efficient Near-Memory Intelligent Reduction," in *HPCA*, 2021.
- [15] H. Aziza, S. Hamdioui, M. Fieback, M. Taouil, M. Moreau, P. Girard, A. Virazel, and K. Coulié, "Multi-Level Control of Resistive RAM (RRAM) Using a Write Termination to Achieve 4 Bits/Cell in High Resistance State," *Electronics*, 2021.
- [16] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The Long-Document Transformer," *arXiv preprint arXiv:2004.05150*, 2020.
- [17] D. Bhattacharjee, R. Devadoss, and A. Chattopadhyay, "ReVAMP: ReRAM based VLIW Architecture for In-memory Computing," in *DATE*, 2017.
- [18] M. N. Bojnordi and E. Ipek, "PARDIS: A Programmable Memory Controller for the DDRx Interfacing Standards," in *ISCA*, 2012.
- [19] Cadence, "Genus Synthesis Solution," https://www.cadence.com/en_US/home/tools/digital-design-and-signoff/synthesis/genus-synthesis-solution.html, 2021, accessed: 2021-11-08.
- [20] Cadence, "Innovus Implementation System," https://www.cadence.com/en_US/home/tools/digital-design-and-signoff/soc-implementation-and-floorplanning/innovus-implementation-system.html, 2021, accessed: 2021-11-08.
- [21] F. Cai, J. M. Correll, S. H. Lee, Y. Lim, V. Bothra, Z. Zhang, M. P. Flynn, and W. Lu, "A Fully Integrated Reprogrammable Memristor-CMOS System for Efficient Multiply-Accumulate Operations," *Nature Electronics*, pp. 290–299, 2019.
- [22] J. Carter, W. Hsieh, L. Stoller, M. Swanson, L. Zhang, E. Brunvand, A. Davis, C.-C. Kuo, R. Kuramkote, M. Parker, L. Schaelicke, and T. Tateyama, "Impulse: Building a Smarter Memory Controller," in *HPCA*, 1999.
- [23] M.-F. Chang, P.-F. Chiu, and S.-S. Sheu, "Circuit Design Challenges in Embedded Memory and Resistive RAM (RRAM) for Mobile SoC and 3D-IC," in *ASP-DAC*, 2011.
- [24] T. Chen, Y. Cheng, Z. Gan, L. Yuan, L. Zhang, and Z. Wang, "Chasing Sparsity in Vision Transformers: An End-to-End Exploration," in *NeurIPS*, 2021.
- [25] T. Chen, Z. Du, N. Sun, J. Wang, C. Wu, Y. Chen, and O. Temam, "DianNao: a small-footprint high-throughput accelerator for ubiquitous machine-learning," in *ASPLOS*, 2014.
- [26] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks," in *ISCA*, 2016.
- [27] Y.-H. Chen, T.-J. Yang, J. Emer, and V. Sze, "Eyeriss v2: A Flexible Accelerator for Emerging Deep Neural Networks on Mobile Devices," *JETCAS*, 2019.
- [28] Y. Chen, T. Luo, S. Liu, S. Zhang, L. He, J. Wang, L. Li, T. Chen, Z. Xu, N. Sun, and O. Temam, "DaDianNao: A Machine-Learning Supercomputer," in *MICRO*, 2014.

- [29] P. Chi, S. Li, C. Xu, T. Zhang, J. Zhao, Y. Liu, Y. Wang, and Y. Xie, "Prime: A Novel Processing-in-Memory Architecture for Neural Network Computation in ReRAM-based Main Memory," in *ISCA*, 2016.
- [30] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating Long Sequences with Sparse Transformers," *arXiv preprint arXiv:1904.10509*, 2019.
- [31] C. Choi, "Multi-Stream Write SSD," *Flash Memory Summit*, 2016.
- [32] K. Choromanski, V. Likhoshershtov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Davis, A. Mohiuddin, L. Kaiser, D. Belanger, L. Colwell, and A. Weller, "Rethinking Attention with Performers," *arXiv preprint arXiv:2009.14794*, 2020.
- [33] G. M. Correia, V. Niculae, and A. F. Martins, "Adaptively Sparse Transformers," *arXiv preprint arXiv:1909.00015*, 2019.
- [34] B. Cui, Y. Li, M. Chen, and Z. Zhang, "Fine-Tune BERT with Sparse Self-Attention Mechanism," in *EMNLP-IJCNLP*, 2019.
- [35] A. Delmas Lascorz, P. Judd, D. M. Stuart, Z. Poulos, M. Mahmoud, S. Sharify, M. Nikolic, K. Siu, and A. Moshovos, "Bit-Tactical: A Software/Hardware Approach to Exploiting Value and Bit Sparsity in Neural Networks," in *ASPLOS*, 2019.
- [36] Q. Deng, L. Jiang, Y. Zhang, M. Zhang, and J. Yang, "DrAcc: A DRAM based Accelerator for Accurate CNN Inference," in *DAC*, 2018.
- [37] R. H. Dennard, F. H. Gaensslen, V. L. Rideout, E. Bassous, and A. R. LeBlanc, "Design of Ion-Implanted MOSFET's with Very Small Physical Dimensions," *JSSC*, 1974.
- [38] X. Dong, C. Xu, Y. Xie, and N. P. Jouppi, "NVSIM: A Circuit-Level Performance, Energy, and Area Model for Emerging Nonvolatile Memory," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 31, no. 7, pp. 994–1007, 2012.
- [39] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," in *ICLR*, 2021.
- [40] C. Eckert, X. Wang, J. Wang, A. Subramanian, R. Iyer, D. Sylvester, D. Blaauw, and R. Das, "Neural Cache: Bit-Serial In-Cache Acceleration of Deep Neural Networks," in *ISCA*, 2018.
- [41] A. Farnahini-Farahani, J. H. Ahn, K. Morrow, and N. S. Kim, "NDA: Near-DRAM Acceleration Architecture Leveraging Commodity DRAM Devices and Standard Memory Modules," in *HPCA*, 2015, pp. 283–295.
- [42] J. Fowers, K. Ovtcharov, M. Papamichael, T. Massengill, M. Liu, D. Lo, S. Alkalay, M. Haselman, L. Adams, M. Ghandi, S. Heil, P. Patel, A. Sapek, G. Weisz, L. Woods, S. Lanka, S. K. Reinhardt, A. M. Caulfield, E. S. Chung, and D. Burger, "A Configurable Cloud-Scale DNN Processor for Real-Time AI," in *ISCA*, 2018.
- [43] S. Froehlich, S. Shirinzadeh, and R. Drechsler, "Parallel Computing of Graph-based Functions in ReRAM," *JETC*, 2022.
- [44] F. Gao, G. Tziantzioulis, and D. Wentzlaff, "ComputeDRAM: In-Memory Compute using Off-the-Shelf DRAMs," in *MICRO*, 2019.
- [45] M. Gao, J. Pu, X. Yang, M. Horowitz, and C. Kozyrakis, "TETRIS: Scalable and Efficient Neural Network Acceleration with 3D Memory," in *ASPLOS*, 2017.
- [46] M. Gao, X. Yang, J. Pu, M. Horowitz, and C. Kozyrakis, "Tangram: Optimized Coarse-Grained Dataflow for Scalable NN Accelerators," in *ASPLOS*, 2019.
- [47] M. Ghasempour, A. Jaleel, J. D. Garside, and M. Luján, "DRaM: Dynamic Re-arrangement of Address Mapping to Improve the Performance of DRAMs," in *MEMSY*, 2016.
- [48] S. Ghodrati, B. H. Ahn, J. Kyung Kim, S. Kinzer, B. R. Yatham, N. Alla, H. Sharma, M. Alian, E. Ebrahimi, N. S. Kim, C. Young, and H. Esmaeilzadeh, "Planaria: Dynamic Architecture Fission for Spatial Multi-Tenant Acceleration of Deep Neural Networks," in *MICRO*, 2020.
- [49] S. Ghodrati, H. Sharma, S. Kinzer, A. Yazdanbakhsh, J. Park, N. S. Kim, D. Burger, and H. Esmaeilzadeh, "Mixed-Signal Charge-Domain Acceleration of Deep Neural networks through Interleaved Bit-Partitioned Arithmetic," in *PACT*, 2020.
- [50] S. Ghodrati, H. Sharma, C. Young, N. S. Kim, and H. Esmaeilzadeh, "Bit-Parallel Vector Composability for Neural Acceleration," in *DAC*, 2020.
- [51] A. Grossi, E. Vianello, M. M. Sabry, M. Barlas, L. Grenouillet, J. Coignus, E. Beigne, T. Wu, B. Q. Le, M. K. Wootters, C. Zambelli, E. Nowak, and S. Mitra, "Resistive RAM Endurance: Array-Level Characterization and Correction Techniques Targeting Deep Learning Applications," *IEEE Transactions on Electron Devices*, vol. 66, no. 3, pp. 1281–1288, 2019.
- [52] Q. Guo, X. Guo, R. Patel, E. Ipek, and E. G. Friedman, "AC-DIMM: Associative Computing with STT-MRAM," in *ISCA*, 2013.
- [53] Y. Halawani, B. Mohammad, M. A. Lebdeh, M. Al-Qutayri, and S. F. Al-Sarawi, "ReRAM-based In-memory Computing for Search Engine and Neural Network Applications," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 2019.
- [54] T. J. Ham, S. J. Jung, S. Kim, Y. H. Oh, Y. Park, Y. Song, J.-H. Park, S. Lee, K. Park, J. W. Lee, and D.-K. Jeong, "A3: Accelerating Attention Mechanisms in Neural Networks with Approximation," in *HPCA*, 2020.
- [55] T. J. Ham, Y. Lee, S. H. Seo, S. Kim, H. Choi, S. J. Jung, and J. W. Lee, "ELSA: Hardware-Software Co-design for Efficient, Lightweight Self-Attention Mechanism in Neural Networks," in *ISCA*, 2021.
- [56] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, "EIE: Efficient Inference Engine on Compressed Deep Neural Network," in *ISCA*, 2016.
- [57] A. Hansson, N. Agarwal, A. Kolli, T. Wenisch, and A. N. Udupi, "Simulating DRAM Controllers for Future System Architecture Exploration," in *ISPASS*, 2014.
- [58] M. He, C. Song, I. Kim, C. Jeong, S. Kim, I. Park, M. Thottethodi, and T. Vijaykumar, "Newton: A DRAM-maker's Accelerator-in-Memory (AiM) Architecture for Machine Learning," in *MICRO*, 2020.
- [59] Z. He, J. Lin, R. Ewetz, J.-S. Yuan, and D. Fan, "Noise Injection Adaption: End-to-end ReRAM Crossbar Mon-Ideal Effect Adaption for Neural Network Mapping," in *DAC*, 2019.
- [60] M. Hu, J. P. Strachan, Z. Li, E. M. Grafals, N. Davila, C. Graves, S. Lam, N. Ge, J. J. Yang, and R. S. Williams, "Dot-product Engine for Neuromorphic Computing: Programming 1T1M Crossbar to Accelerate Matrix-Vector Multiplication," in *DAC*, 2016.
- [61] S. Hussain, M. Javaheripi, P. Neekhar, R. Kastner, and F. Koushanfar, "FastWave: Accelerating Autoregressive Convolutional Neural Networks on FPGA," in *ICCAD*, 2019.
- [62] M. Imani, S. Gupta, Y. Kim, and T. Rosing, "FloatPIM: In-Memory Acceleration of Deep Neural Network Training with High Precision," in *ISCA*, 2019.
- [63] L. Jiang, M. Kim, W. Wen, and D. Wang, "XNOR-POP: A Processing-In-Memory Architecture for Binary Convolutional Neural Networks in Wide-IO2 DRAMs," in *ISLPED*, 2017.
- [64] B. K. Joardar, J. R. Doppa, P. P. Pande, H. Li, and K. Chakrabarty, "AccuReD: High Accuracy Training of CNNs on ReRAM/GPU Heterogeneous 3-D Architecture," *IEEE TCAD*, 2021.
- [65] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P.-I. Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmaghami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, D. Killebrew, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snellman, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, and D. H. Yoon, "In-Datacenter Performance Analysis of a Tensor Processing Unit," in *ISCA*, 2017.
- [66] P. Judd, J. Albericio, T. Hetherington, T. M. Aamodt, and A. Moshovos, "Stripes: Bit-serial Deep Neural Network Computing," in *MICRO*, 2016.

- [67] M. Kang, S. K. Gonugondla, A. Patil, and N. R. Shanbhag, "A Multi-functional In-memory Inference Processor using a Standard 6T SRAM Array," *IEEE Journal of Solid-State Circuits*, 2018.
- [68] M. Kang, M.-S. Keel, N. R. Shanbhag, S. Eilert, and K. Curewitz, "An Energy-Efficient VLSI Architecture for Pattern Recognition via Deep Embedding of Computation in SRAM," in *ICASSP*, 2014.
- [69] Y. Kang, W. Huang, S.-M. Yoo, D. Keen, Z. Ge, V. Lam, P. Pattnaik, and J. Torrellas, "FlexRAM: Toward an Advanced Intelligent Memory System," in *ICCR*, 1999.
- [70] S. Kao, S. Subramanian, G. Agrawal, A. Yazdanbakhsh, and T. Krishna, "FLAT: An Optimized Dataflow for Mitigating Attention Bottlenecks," *arXiv preprint arXiv:2107.06419*, 2021.
- [71] J. D. M.-W. C. Kenton and L. K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *NAACL-HLT*, 2019.
- [72] B. Kim, J. Chung, E. Lee, W. Jung, S. Lee, J. Choi, J. Park, M. Wi, S. Lee, and J. H. Ahn, "MViD: Sparse Matrix-Vector Multiplication in Mobile DRAM for Accelerating Recurrent Neural Networks," *IEEE Transactions on Computers*, 2020.
- [73] D. Kim, J. Kung, S. Chai, S. Yalamanchili, and S. Mukhopadhyay, "Neurocube: A Programmable Digital Neuromorphic Architecture with High-Density 3D Memory," in *ISCA*, 2016.
- [74] H. Kim, H. Park, T. Kim, K. Cho, E. Lee, S. Ryu, H.-J. Lee, K. Choi, and J. Lee, "GradPIM: A Practical Processing-in-DRAM Architecture for Gradient Descent," in *HPCA*, 2021.
- [75] S. Kim, S. Shen, D. Thorsley, A. Gholami, J. Hassoun, and K. Keutzer, "Learned Token Pruning for Transformers," *arXiv preprint arXiv:2107.00910*, 2021.
- [76] N. Kitaev, L. Kaiser, and A. Levskaya, "Reformer: The Efficient Transformer," *arXiv preprint arXiv:2001.04451*, 2020.
- [77] P. M. Kogge, "EXECUBE-A New Architecture for Scaleable MPPs," in *ICPP*, 1994.
- [78] A. Krizhevsky and G. Hinton, "Learning Multiple Layers of Features from Tiny Images," *Computer Science Department, University of Toronto, Tech. Rep*, 2009.
- [79] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, "Understanding Reuse, Performance, and Hardware Cost of DNN Dataflow: A Data-Centric Approach," in *MICRO*, 2019.
- [80] H. Kwon, A. Samajdar, and T. Krishna, "MAERI: Enabling Flexible Dataflow Mapping over DNN Accelerators via Reconfigurable Interconnects," in *ASPLOS*, 2018.
- [81] Y. Kwon, Y. Lee, and M. Rhu, "TensorDIMM: A Practical Near-Memory Processing Architecture for Embeddings and Tensor Operations in Deep Learning," in *MICRO*, 2019.
- [82] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soiccut, "ALBERT: A Lite BERT for Self-supervised Learning of Language Representations," in *ICLR*, 2019.
- [83] D. Lee, S. Kang, and K. Choi, "CompPEND: Computation Pruning through Early Negative Detection for ReLU in a deep neural network accelerator," in *ICS*, 2018.
- [84] J. Lee, J. H. Ahn, and K. Choi, "Buffered Compares: Excavating the Hidden Parallelism Inside DRAM Architectures with Lightweight Logic," in *DATE*, 2016.
- [85] J. Lee, C. Kim, S. Kang, D. Shin, S. Kim, and H.-J. Yoo, "UNPU: A 50.6 TOPS/W Unified Deep Neural Network Accelerator with 1b-to-16b Fully-Variable Weight Bit-Precision," in *ISSCC*, 2018.
- [86] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, O. Seongil, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, "Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology: Industrial Product," in *ISCA*, 2021.
- [87] S. Li, D. Niu, K. T. Malladi, H. Zheng, B. Brennan, and Y. Xie, "DRISA: A DRAM-based Reconfigurable In-Situ Accelerator," in *MICRO*, 2017.
- [88] S. Li, C. Xu, Q. Zou, J. Zhao, Y. Lu, and Y. Xie, "Pinatubo: A Processing-in-Memory Architecture for Bulk Bitwise Operations in Emerging Non-Volatile Memories," in *DAC*, 2016.
- [89] W. Li, P. Xu, Y. Zhao, H. Li, Y. Xie, and Y. Lin, "TIMELY: Pushing Data Movements and Interfaces in PIM Accelerators towards Local and in Time Domain," in *ISCA*, 2020.
- [90] Z. Li, S. Ghodrati, A. Yazdanbakhsh, H. Esmaeilzadeh, and M. Kang, "Accelerating Attention through Gradient-Based Learned Runtime Pruning," in *ISCA*, 2022.
- [91] Y. Lin, C. Sakr, Y. Kim, and N. Shanbhag, "PredictiveNet: An Energy-Efficient Convolutional Neural Network via Zero Prediction," in *ISCA*, 2017.
- [92] J. Liu, H. Zhao, M. A. Ogleari, D. Li, and J. Zhao, "Processing-in-Memory for Energy-Efficient Neural Network Training: A Heterogeneous Approach," in *MICRO*, 2018.
- [93] P. J. Liu, M. Saleh, E. Pot, B. Goodrich, R. Sepassi, L. Kaiser, and N. Shazeer, "Generating Wikipedia by Summarizing Long Sequences," *arXiv preprint arXiv:1801.10198*, 2018.
- [94] S. Liu, Z. Du, J. Tao, D. Han, T. Luo, Y. Xie, Y. Chen, and T. Chen, "Cambricon: An Instruction Set Architecture for Neural Networks," in *ISCA*, 2016.
- [95] Q. Lou, W. Wen, and L. Jiang, "3DICT: A Reliable and QoS Capable Mobile Process-in-Memory Architecture for Lookup-Based CNNs in 3D XPoint ReRAMs," in *ICCAD*, 2018.
- [96] L. Lu, Y. Jin, H. Bi, Z. Luo, P. Li, T. Wang, and Y. Liang, "Sanger: A Co-Design Framework for Enabling Sparse Attention using Reconfigurable Architecture," in *MICRO*, 2021.
- [97] K. Mai, T. Paaske, N. Jayasena, R. Ho, W. J. Dally, and M. Horowitz, "Smart Memories: A Modular Reconfigurable Architecture," in *ISCA*, 2000.
- [98] H. Mao, M. Song, T. Li, Y. Dai, and J. Shu, "LerGAN: A Zero-free, Low Data Movement and PIM-based GAN Architecture," in *MICRO*, 2018.
- [99] S. Merity, "The WikiText Long Term Dependency Language Modeling Dataset," <https://blog.salesforceairesearch.com/the-wikitext-long-term-dependency-language-modeling-dataset/>, 2021, accessed: 2021-11-08.
- [100] P. Michel, O. Levy, and G. Neubig, "Are Sixteen Heads Really Better than One?" *arXiv preprint arXiv:1905.10650*, 2019.
- [101] S. Mittal, "A Survey of ReRAM-based Architectures for Processing-In-Memory and Neural Networks," *Machine learning and knowledge extraction*, 2018.
- [102] D. Niu, C. Xu, N. Muralimanohar, N. P. Jouppi, and Y. Xie, "Design Trade-Offs for High Density Cross-Point Resistive Memory," in *ISLPED*, 2012.
- [103] A. Nowatzky, F. Pong, and A. Saulsbury, "Missing the Memory Wall: The Case for Processor/Memory Integration," in *ISCA*, 1996.
- [104] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally, "SCNN: An Accelerator for Compressed-sparse Convolutional Neural Networks," in *ISCA*, 2017.
- [105] J. Park, H. Yoon, D. Ahn, J. Choi, and J.-J. Kim, "OPTIMUS: OPTImized matrix MULTIplication Structure for Transformer neural network accelerator," in *MLSys*, 2020.
- [106] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *NeurIPS*, 2019.
- [107] D. Patterson, T. Anderson, N. Cardwell, R. Fromm, K. Keeton, C. Kozyrakis, R. Thomas, and K. Yelick, "A Case for Intelligent RAM," *IEEE Micro*, 1997.
- [108] S. H. Pugsley, J. Jests, H. Zhang, R. Balasubramanian, V. Srinivasan, A. Buyuktosunoglu, A. Davis, and F. Li, "NDC: Analyzing the Impact of 3D-Stacked Memory+Logic Devices on MapReduce Workloads," in *ISPASS*, 2014.
- [109] E. Qin, A. Samajdar, H. Kwon, V. Nadella, S. Srinivasan, D. Das, B. Kaul, and T. Krishna, "SIGMA: A Sparse and Irregular GEMM Accelerator with Flexible Interconnects for DNN Training," in *HPCA*, 2020.

- [110] J. Qiu, H. Ma, O. Levy, S. W.-t. Yih, S. Wang, and J. Tang, "Blockwise Self-Attention for Long Document Understanding," *arXiv preprint arXiv:1911.02972*, 2019.
- [111] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," *OpenAI blog*, 2019.
- [112] J. W. Rae, A. Potapenko, S. M. Jayakumar, and T. P. Lillicrap, "Compressive transformers for long-range sequence modelling," *arXiv preprint arXiv:1911.05507*, 2019.
- [113] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQUAD: 100,000+ Questions for Machine Comprehension of Text," *arXiv preprint arXiv:1606.05250*, in *ISCA*, 2016.
- [114] M. Ramezani, N. Elyasi, M. Arjomand, M. T. Kandemir, and A. Sivasubramanian, "Exploring the Impact of Memory Block Permutation on Performance of a Crossbar ReRAM Main Memory," in *IIISWC*, 2017, pp. 167–176.
- [115] B. Reagen, P. Whatmough, R. Adolf, S. Rama, H. Lee, S. K. Lee, J. M. Hernández-Lobato, G.-Y. Wei, and D. Brooks, "Minerva: Enabling Low-Power, Highly-Accurate Deep Neural Network Accelerators," in *ISCA*, 2016.
- [116] B. D. Rouhani, M. Samragh, M. Javaheripi, T. Javidi, and F. Koushanfar, "DeepFense: Online Accelerated Defense against Adversarial Deep Learning," in *ICCAD*, 2018.
- [117] A. Roy, M. Saffar, A. Vaswani, and D. Grangier, "Efficient Content-based Sparse Attention with Routing Transformers," *Transactions of the Association for Computational Linguistics*, 2021.
- [118] S. Ryu, H. Kim, W. Yi, and J.-J. Kim, "BitBlade: Area and Energy-Efficient Precision-Scalable Neural Network Accelerator with Bitwise Summation," in *DAC*, 2019.
- [119] M. Samragh, M. Javaheripi, and F. Koushanfar, "EncoDeep: Realizing Bit-Flexible Encoding for Deep Neural Networks," *TECS*, 2019.
- [120] V. Seshadri, Y. Kim, C. Fallin, D. Lee, R. Ausavarungrun, G. Pekhimenko, Y. Luo, O. Mutlu, P. B. Gibbons, M. A. Kozuch, and T. C. Mowry, "RowClone: Fast and Energy-Efficient In-DRAM Bulk Data Copy and Initialization," in *MICRO*, 2013.
- [121] V. Seshadri, D. Lee, T. Mullins, H. Hassan, A. Boroumand, J. Kim, M. A. Kozuch, O. Mutlu, P. B. Gibbons, and T. C. Mowry, "Ambit: In-Memory Accelerator for Bulk Bitwise Operations using Commodity DRAM Technology," in *MICRO*. IEEE, 2017.
- [122] V. Seshadri, T. Mullins, A. Boroumand, O. Mutlu, P. B. Gibbons, M. A. Kozuch, and T. C. Mowry, "Gather-Scatter DRAM: In-DRAM Address Translation to Improve the Spatial Locality of Non-unit Strided Accesses," in *MICRO*, 2015.
- [123] A. Shafiee, A. Nag, N. Muralimanohar, R. Balasubramanian, J. P. Strachan, M. Hu, R. S. Williams, and V. Srikumar, "ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars," in *ISCA*, 2016.
- [124] Y. S. Shao, J. Clemons, R. Venkatesan, B. Zimmer, M. Fojtik, N. Jiang, B. Keller, A. Klinefelter, N. Pinckney, P. Raina, S. G. Tell, Y. Zhang, W. J. Dally, J. Emer, C. T. Gray, B. Khailany, and S. W. Keckler, "Simba: Scaling Deep-Learning Inference with Multi-Chip-Module-Based Architecture," in *MICRO*, 2019.
- [125] S. Sharify, A. D. Lascorz, M. Mahmoud, M. Nikolic, K. Siu, D. M. Stuart, Z. Poulos, and A. Moshovos, "Laconic Deep Learning Inference Acceleration," in *ISCA*, 2019.
- [126] S. Sharify, A. D. Lascorz, K. Siu, P. Judd, and A. Moshovos, "Loom: Exploiting Weight and Activation Precisions to Accelerate Convolutional Neural Networks," in *DAC*, 2018.
- [127] H. Sharma, J. Park, D. Mahajan, E. Amaro, J. Kim, C. Shao, A. Misra, and H. Esmaeilzadeh, "From High-Level Deep Neural Models to FPGAs," in *MICRO*, 2016.
- [128] H. Sharma, J. Park, N. Suda, L. Lai, B. Chau, V. Chandra, and H. Esmaeilzadeh, "Bit Fusion: Bit-Level Dynamically Composable Architecture for Accelerating Deep Neural Networks," in *ISCA*, 2018.
- [129] H. Shin, D. Kim, E. Park, S. Park, Y. Park, and S. Yoo, "McDRAM: Low latency and energy-efficient matrix computations in DRAM," *IEEE TCAD*, 2018.
- [130] G. Shomron, R. Banner, M. Shkolnik, and U. Weiser, "Thanks for Nothing: Predicting Zero-valued Activations with Lightweight Convolutional Neural Networks," in *ECCV*, 2020.
- [131] L. Song, X. Qian, H. Li, and Y. Chen, "PipeLayer: A Pipelined ReRAM-based Accelerator for Deep Learning," in *HPCA*, 2017.
- [132] L. Song, Y. Zhuo, X. Qian, H. Li, and Y. Chen, "GraphR: Accelerating Graph Processing using ReRAM," in *HPCA*, 2018.
- [133] R. St. Amant, A. Yazdanbakhsh, J. Park, B. Thwaites, H. Esmaeilzadeh, A. Hassibi, L. Ceze, and D. Burger, "General-Purpose Code Acceleration with Limited-Precision Analog Computation," in *ISCA*, 2014.
- [134] J. A. Starzyk and Basawaraj, "Memristor Crossbar Architecture for Synchronous Neural Networks," *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2014.
- [135] J. R. Stevens, R. Venkatesan, S. Dai, B. Khailany, and A. Raghunathan, "Softmax: Hardware/Software Co-Design of an Efficient Softmax for Transformers," *arXiv preprint arXiv:2103.09301*, 2021.
- [136] A. Stojcevski, H. P. Le, J. Singh, and A. Zayegh, "Flash ADC architecture," *Electronics letters*, vol. 39, no. 6, pp. 501–502, 2003.
- [137] T. Tambe, C. Hooper, L. Pentecost, T. Jia, E.-Y. Yang, M. Donato, V. Sanh, P. Whatmough, A. M. Rush, D. Brooks *et al.*, "EdgeBERT: Sentence-level Energy Optimizations for Latency-Aware Multi-Task NLP Inference," in *MICRO*, 2021.
- [138] Y. Tay, M. Dehghani, S. Abnar, Y. Shen, D. Bahri, P. Pham, J. Rao, L. Yang, S. Ruder, and D. Metzler, "Long Range Arena: A Benchmark for Efficient Transformers," *arXiv preprint arXiv:2011.04006*, 2020.
- [139] Transformers: Opening New Age of Artificial Intelligence Ahead, "ADC performance survey 1997-2016," <https://www.analyticsinsight.net/transformers-opening-new-age-of-artificial-intelligence-ahead/>, 2021, accessed: 2022-04-21.
- [140] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is All You Need," in *NeurIPS*, 2017.
- [141] W. Wan, R. Kubendran, S. B. Eryilmaz, W. Zhang, Y. Liao, D. Wu, S. Deiss, B. Gao, P. Raina, S. Joshi, H. Wu, G. Cauwenberghs, and H.-S. P. Wong, "A 74 TMACS/W CMOS-RRAM Neurosynaptic Core with Dynamically Reconfigurable Dataflow and In-situ Transposable Weights for Probabilistic Graphical Models," in *ISSCC*, 2020.
- [142] W. Wan, R. Kubendran, C. Schaefer, S. B. Eryilmaz, W. Zhang, D. Wu, S. Deiss, P. Raina, H. Qian, B. Gao, S. Joshi, H. Wu, H. S. P. Wong, and G. Cauwenberghs, "Edge AI without Compromise: Efficient, Versatile and Accurate Neurocomputing in Resistive Random-Access Memory," *arXiv preprint arXiv:2108.07879*, 2021.
- [143] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. R. Bowman, "GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding," *arXiv preprint arXiv:1804.07461*, 2018.
- [144] H. Wang, Z. Zhang, and S. Han, "SpAtten: Efficient Sparse Attention Architecture with Cascade Token and Head Pruning," in *HPCA*, 2021.
- [145] Y. Wang, Z. Zhu, F. Chen, M. Ma, G. Dai, Y. Wang, H. Li, and Y. Chen, "Rerac: In-ReRAM Acceleration with Access-Aware Mapping for Personalized Recommendation," in *ICCAD*, 2021.
- [146] Z. Wang, J. Wohlwend, and T. Lei, "Structured Pruning of Large Language Models," *arXiv preprint arXiv:1910.04732*, 2019.
- [147] W. Wen, Y. He, S. Rajbhandari, M. Zhang, W. Wang, F. Liu, B. Hu, Y. Chen, and H. Li, "Learning Intrinsic Sparse Structures within Long Short-Term Memory," *arXiv preprint arXiv:1709.05027*, 2017.
- [148] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "HuggingFace's Transformers: State-of-the-Art Natural Language Processing," *arXiv preprint arXiv:1910.03771*, 2019.
- [149] M.-C. Wu, W.-Y. Jang, C.-H. Lin, and T.-Y. Tseng, "A Study on Low-Power, Nanosecond Operation and Multilevel Bipolar Resistance Switching in Ti/ZrO₂/Pt Nonvolatile Memory with 1T1R Architecture," *Semiconductor Science and Technology - SEMICONDUCTOR SCI TECHNOL*, 2012.
- [150] T. Yang, D. Li, Y. Han, Y. Zhao, F. Liu, X. Liang, Z. He, and L. Jiang, "PIMGCN: A ReRAM-Based PIM Design for Graph

- Convolutional Network Acceleration,” in *DAC*, 2021.
- [151] P. Yao, H. Wu, B. Gao, S. B. Eryilmaz, X. Huang, W. Zhang, Q. Zhang, N. Deng, L. Shi, H.-S. P. Wong *et al.*, “Face Classification Using Electronic Synapses,” *Nature communications*, vol. 8, p. 15199, 2017.
- [152] A. Yazdanbakhsh, M. Brzozowski, B. Khaleghi, S. Ghodrati, K. Samadi, N. S. Kim, and H. Esmailzadeh, “FlexiGAN: An End-to-End Solution for FPGA Acceleration of Generative Adversarial Networks,” in *FCCM*, 2018.
- [153] A. Yazdanbakhsh, H. Falahati, P. J. Wolfe, K. Samadi, H. Esmailzadeh, and N. S. Kim, “GANAX: A Unified SIMD-MIMD Acceleration for Generative Adversarial Network,” in *ISCA*, 2018.
- [154] A. Yazdanbakhsh, J. Park, H. Sharma, P. Lotfi-Kamran, and H. Esmailzadeh, “Neural Acceleration for GPU Throughput Processors,” in *MICRO*, 2015.
- [155] A. Yazdanbakhsh, K. Seshadri, B. Akin, J. Laudon, and R. Narayanaswami, “An Evaluation of Edge TPU Accelerators for Convolutional Neural Networks,” *arXiv preprint arXiv:2102.10423*, 2021.
- [156] A. Yazdanbakhsh, C. Song, J. Sacks, P. Lotfi-Kamran, H. Esmailzadeh, and N. S. Kim, “In-DRAM Near-Data Approximate Acceleration for GPUs,” in *PACT*, 2018.
- [157] D. Ye, Y. Lin, Y. Huang, and M. Sun, “TR-BERT: Dynamic Token Reduction for Accelerating BERT Inference,” *arXiv preprint arXiv:2105.11618*, 2021.
- [158] Z. Ye, Q. Guo, Q. Gan, X. Qiu, and Z. Zhang, “BP-Transformer: Modelling Long-Range Context via Binary Partitioning,” *arXiv preprint arXiv:1911.04070*, 2019.
- [159] S. Yin, Z. Jiang, J.-S. Seo, and M. Seok, “XNOR-SRAM: In-Memory Computing SRAM Macro for Binary/Ternary Deep Neural Networks,” *IEEE Journal of Solid-State Circuits*, 2020.
- [160] S. Yu, Y. Wu, and H.-S. P. Wong, “Investigating the Switching Dynamics and Multilevel Capability of Bipolar Metal Oxide Resistive Switching Memory,” *Applied Physics Letters*, 2011.
- [161] G. Yuan, P. Behnam, Z. Li, A. Shafiee, S. Lin, X. Ma, H. Liu, X. Qian, M. N. Bojnordi, Y. Wang, and C. Ding, “FORMS: Fine-grained Polarized ReRAM-based In-situ Computation for Mixed-signal DNN Accelerator,” in *ISCA*, 2021.
- [162] A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos, “GOBO: Quantizing Attention-based NLP Models for Low Latency and Energy Efficient Inference,” in *MICRO*, 2020.
- [163] M. Zaheer, G. Guruganesh, A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, and A. Ahmed, “Big Bird: Transformers for Longer Sequences,” in *NeurIPS*, 2020.
- [164] D. Zhang, N. Jayasena, A. Lyashevsky, J. L. Greathouse, L. Xu, and M. Ignatowski, “TOP-PIM: Throughput-Oriented Programmable Processing in Memory,” in *HPDC*, 2014.
- [165] L. Zhang, D. Strukov, H. Saadeldeen, D. Fan, M. Zhang, and D. Franklin, “SpongeDirectory: Flexible Sparse Directories Utilizing Multi-Level Memristors,” in *PACT*, 2014.
- [166] M. Zhang, Y. Zhuo, C. Wang, M. Gao, Y. Wu, K. Chen, C. Kozyrakis, and X. Qian, “GraphP: Reducing Communication for PIM-based Graph Processing with Efficient Data Partition,” in *HPCA*, 2018.
- [167] S. Zhang, Z. Du, L. Zhang, H. Lan, S. Liu, L. Li, Q. Guo, T. Chen, and Y. Chen, “Cambricon-X: An Accelerator for Sparse Neural Networks,” in *MICRO*, 2016.
- [168] G. Zhao, J. Lin, Z. Zhang, X. Ren, Q. Su, and X. Sun, “Explicit Sparse Transformer: Concentrated Attention through Explicit Selection,” *arXiv preprint arXiv:1912.11637*, 2019.
- [169] Y. Zhuo, C. Wang, M. Zhang, R. Wang, D. Niu, Y. Wang, and X. Qian, “GraphQ: Scalable PIM-based Graph Processing,” in *MICRO*, 2019.